

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf\_e5140909-890\agent-afbab6d6a62829d2c.jsonl`
- SHA-256: `50f4d7f9abbf5da6c0c7126b204021da6876da5be9d9fc30e66c5d222e069b71`
- Source modified: `2026-06-18T07:57:27+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf\_e5140909-890`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`

Transcript

1. user (2026-06-18T07:54:35.123Z)

Work READ-ONLY in this git worktree pinned to origin/master (DO NOT modify):

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base
Read the ACTUAL code (and the matching tests/ file) before asserting anything; never guess.
You are a senior engineer scoping a SAFE refactoring effort. The owner wants critical tech debt cleaned up in files that "grew a lot", with ZERO behavior change and NO quality regression. Propose only refactors that are: (a) behavior-PRESERVING (pure moves/extractions/dedup/dead-code removal ? NOT redesigns, NOT logic changes), (b) backed by EXISTING tests that would catch a regression (assess this honestly per proposal ? if coverage is weak/none, say so and rate risk higher), (c) cleanly separable into ONE reviewable PR. For each: the exact seam (what moves where), the behavior-preservation argument, honest test_coverage, risk, est diff size, value. PREFER high-value + low-risk + well-tested. It is FINE to say a giant function should NOT be refactored yet because coverage is too thin ? flag that as a 'needs tests first' note in debt.

Signals (LOC / longest-functions): gemini.process_video=434L/71br, yolo_hybrid=309L, trajectory_hybrid=231L, motion=204L, compiler.compile_highlights=202L, golden_fixtures=1113L (freeze_real_fixture=186L), main.py=871L (main=168L, _execute_processing=112L), database.py=738L (_init_db=187L), wasb_infer=699L (run=185L), ablation=620L, experiment=579L, tracknet_runner=506L.

CLUSTER: core ? backend/main.py(871) and backend/infrastructure/database.py(738). main.py mixes FastAPI routes + CLI + processing; database.py is one big class. Propose ONLY low-risk splits (e.g. extract a cohesive helper module, group methods) with strong existing coverage. Be conservative ? these are hot paths; if a split risks import cycles or app-wiring breakage, rate risk high and say so.

2. assistant (2026-06-18T07:54:38.313Z)

I'll scope safe refactors for the `core` cluster: `backend/main.py` and `backend/infrastructure/database.py`. Let me read the actual code and its tests first.

3. user (2026-06-18T07:54:39.744Z)

```
871 backend/main.py
738 backend/infrastructure/database.py
1609 total
---TESTS---
__init__.py
__pycache__
test_ablation.py
test_annotation_providers.py
test_api_endpoints.py
test_async_jobs.py
test_audio_features.py
```

test_automated_provider.py
test_base.py
test_calibrate_local.py
test_calibrate_wasb.py
test_calibration.py
test_cleanup_caches.py
test_compiler.py
test_config.py
test_config_wasb.py
test_database.py
test_detector_keying.py
test_distill_local.py
test_eval_annotations.py
test_eval_batch.py
test_eval_classifier.py
test_eval_harness.py
test_eval_metrics.py
test_eval_partial_labels.py
test_eval_regression.py
test_eval_stats.py
test_eval_training.py
test_execution_policy.py
test_experiment.py
test_experiment_honest.py
test_export_wasb_segments.py
test_fusion_compare.py
test_fusion_features.py
test_fusion_golden_telemetry.py
test_fusion_hybrid.py
test_gemini.py
test_gemini_refine.py
test_generate_highlights.py
test_geometry.py
test_golden_fixtures.py
test_golden_real_fixtures.py
test_gt_loader.py
test_hashing.py
test_hit_detector.py
test_import_smoke.py
test_job_self_scheduling.py
test_job_worker_loop.py
test_logging_setup.py
test_nightly_regression.py
test_path_safety.py
test_per_user_eval.py
test_per_user_gate.py
test_per_user_regression.py
test_person_detector.py
test_personalization_serving.py
test_promotion.py
test_provider_gemini.py
test_providers.py
test_rally_gate.py
test_rally_seg_eval.py
test_rally_seq_proto.py
test_rally_slicer.py
test_rally_summary.py
test_reingest_safety.py
test_reliability_hardening.py
test_reporting.py
test_reporting_harvest.py
test_reporting_spec_faq.py
test_results.py
test_run_signals.py
test_run_telemetry.py

```
test_score_calibration.py
test_segmentation_metrics.py
test_serve_contrast.py
test_served_gate_a_regression.py
test_splits.py
test_sport_profiles.py
test_tolerance_metrics.py
test_tracknet_hybrid.py
```

4. assistant (2026-06-18T07:54:43.304Z)

Let me read both files and their test files.

5. user (2026-06-18T07:54:44.512Z)

```
1     import os
2     import sys
3     import argparse
4     import logging
5     import threading
6     import traceback
7     import uuid
8     from concurrent.futures import ThreadPoolExecutor
9     import uvicorn
10    from dotenv import load_dotenv
11
12    load_dotenv()
13
14    # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
15    logging.basicConfig(
16        level=logging.INFO,
17        format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
18        datefmt='%H:%M:%S'
19    )
20    logger = logging.getLogger(__name__)
21    from fastapi import FastAPI, HTTPException, Request
22    from fastapi.middleware.cors import CORSMiddleware
23    from pydantic import BaseModel
24    from typing import List, Dict, Any, Optional
25
26    from fastapi.staticfiles import StaticFiles
27    from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
28    from backend.pipeline.validators.base import registry
29    from backend.infrastructure.database import Database
30    from backend.pipeline.segmenters.base import segmenter_registry
31    from backend.pipeline.stitcher.compiler import VideoCompiler
32    from backend.utils.hashing import compute_video_id # canonical file-identity hashing
33    from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
34    from backend.config import load_config as load_app_config, AppConfig, StitchingConfig #
new typed config
35    from backend.results import Failure # standardized error/result contract
36    from backend.reporting import emit_indexer_report
37
38    app = FastAPI(title="Sports Highlight Indexer API")
39
40    # Enable CORS for the local web interface. allow_origins='' with allow_credentials=True
is
41    # rejected by browsers per the fetch spec and is an unsafe default to carry into the
42    # auth/tenancy work; this tool uses no cookies, so credentials stay off.
43    app.add_middleware(
44        CORSMiddleware,
45        allow_origins=["*"],
46        allow_credentials=False,
```

```

47     allow_methods=["*"],
48     allow_headers=["*"],
49 )
50
51 # Serves static frontend files directly (now under api/ per approved structure)
52 os.makedirs("backend/api/static", exist_ok=True)
53 app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
54
55 @app.get("/", response_class=HTMLResponse)
56 def serve_index():
57     index_path = "backend/api/static/index.html"
58     if os.path.exists(index_path):
59         with open(index_path, "r", encoding="utf-8") as f:
60             return f.read()
61     return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
62
63 # Global variables initialized at startup
64 db_instance: Optional[Database] = None
65 global_config: Optional[AppConfig] = None # now typed (Pydantic model)
66
67 # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
68 # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
69 # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
70 # already parallelize their AI handoff internally via their own pools.
71 _job_executor: Optional[ThreadPoolExecutor] = None
72
73 def _get_executor() -> ThreadPoolExecutor:
74     """Lazy module-global executor; tests monkeypatch this for inline execution."""
75     global _job_executor
76     if _job_executor is None:
77         _job_executor = ThreadPoolExecutor(max_workers=1,
thread_name_prefix="jobworker")
78     return _job_executor
79
80 # Legacy thin wrapper for any remaining direct calls during transition.
81 # New code should import load_app_config / AppConfig from backend.config directly.
82 def load_config(config_path: str = "config.json") -> Dict[str, Any]:
83     cfg = load_app_config(config_path)
84     return cfg.model_dump(mode="json")
85
86 # --- API Models ---
87 class ProcessRequest(BaseModel):
88     video_path: str
89     player_pool: Optional[List[str]] = None
90     max_frames: Optional[int] = None
91     segmenter: Optional[str] = None
92
93 class QueryRequest(BaseModel):
94     video_id: str
95     server: Optional[str] = None
96     receiver: Optional[str] = None
97     duration_min: Optional[float] = None
98     event_type: Optional[str] = None
99
100 class CompileRequest(BaseModel):
101     video_id: str
102     segment_ids: List[int]
103     output_filename: str
104
105 def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
106     """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
107
108     On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
109     fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)

```

```

and
110     keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
111     """
112     if segments:
113         db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
114     else:
115         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
116
117
118     # --- API Endpoints ---
119     @app.get("/api/config")
120     def get_config():
121         return global_config
122
123     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
124         """The full hash ? validate ? segment ? report pipeline for one video.
125
126         This is the former synchronous /api/process body, unchanged in behavior, now run on
127         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
128         the sync endpoint used to return (UI compatibility); the per-video observability
129         report is still emitted in `finally:` and grafted as response["reports"].
130
131         `progress` is an optional callable(stage: str) used to surface coarse job progress.
132         Raises on unexpected internal errors ? the caller records those as a job failure
133         (after this function has already restored the pre-run segments, see below).
134         """
135         notify = progress or (lambda stage: None)
136
137         notify("hashing")
138         video_id = compute_video_id(req.video_path)
139         was_reingest = db_instance.get_video(video_id) is not None
140
141         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
142         notify("validating")
143         logger.info(f"Running validations for {req.video_path}...")
144         val_results, val_context = registry.run_all_with_context(req.video_path,
global_config)
145         failures = [r for r in val_results if not r.passed]
146
147         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
148         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
149         seg_name = req.segmenter or default_seg
150         segmenter_actual = None
151         segmentation_ran = False
152         response: Optional[Dict[str, Any]] = None
153         try:
154             if failures:
155                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
156                 # status; it no longer destroys the video's prior good segments.
157                 db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
158                 response = {
159                     "video_id": video_id,
160                     "status": "failed",
161                     "message": "Video failed validation checks.",
162                     "results": [r.model_dump() for r in val_results],
163                 }
164                 return response
165
166             # 2. Run segmenter & indexer
167             logger.info("[API] Video passed checks. Segmenting and indexing...")
168             db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for

```

```

r in val_results])
169
170     segmenter_cls = segmenter_registry.get(seg_name)
171     if not segmenter_cls:
172         # Submit-time validation already 400s unknown names; this is the defensive
173         # in-worker path (e.g. config default pointing at an unregistered
segmenter).
174         available = list(segmenter_registry.list_available().keys())
175         response = {
176             "video_id": video_id,
177             "status": "failed",
178             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
179             "results": [r.model_dump() for r in val_results],
180             "play_segments": [],
181         }
182         return response
183
184     logger.info(f"[API] Using segmenter: {seg_name}")
185     notify(f"segmenting ({seg_name})")
186     segmenter_actual = seg_name
187     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
188     # if the run yields segments; otherwise drop the new partial rows and keep the
old.
189     play_wm, cand_wm = db_instance.segment_watermarks(video_id)
190     segmenter = segmenter_cls(db_instance, global_config)
191     try:
192         result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
193     except Exception:
194         # A raising segmenter must not leave half-written rows: restore the pre-run
195         # state (same destroy-after-confirm contract as the Failure branch), then
let
196         # the job worker record the failure.
197         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
198         raise
199     segmentation_ran = True
200
201     if isinstance(result, Failure):
202         logger.error(f"[API] Segmenter failed: {result.message}")
203         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
204         response = {
205             "video_id": video_id,
206             "status": "failed",
207             "message": f"Segmenter '{seg_name}' failed: {result.message}",
208             "results": [r.model_dump() for r in val_results],
209             "play_segments": [],
210         }
211         return response
212
213     segments = result.value if hasattr(result, "value") else result
214     notify("finalizing")
215     _finalize_reingest(video_id, segments, play_wm, cand_wm)
216
217     response = {
218         "video_id": video_id,
219         "status": "success",
220         "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
221         "results": [r.model_dump() for r in val_results],
222         "play_segments": segments,
223     }
224     return response
225 finally:

```

```

226         # Always emit the per-video observability report (next to the video, or to
227         # report.output_dir). Never raises. Surface the paths in the response.
228         paths = emit_indexer_report(
229             db=db_instance, video_id=video_id, video_path=req.video_path,
230             config=global_config,
231             val_results=val_results, val_context=val_context,
232             segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
233             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
234             )
235         if paths and isinstance(response, dict):
236             response["reports"] = paths
237
238     class JobWaiting(Exception):
239         """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
240         WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`",
241         NOT a failure. The worker persists `next_poll_at` + `progress` (via
242         `set_job_waiting`),
243         releases the thread, and re-claims the job when due (`claim_due_job`). No work to
244         date
245         is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
246         contract.
247         The wiring is additive: the production segmenter path never raises this today (zero
248         behaviour change), so a job runs exactly as before. The hook is what a later slice
249         (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
250         """
251         def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
252             super().__init__(f"job parked for {delay_sec:.0f}s")
253             self.delay_sec = max(0.0, float(delay_sec))
254             self.progress = progress
255
256     def _job_to_request(job: Dict[str, Any]) -> ProcessRequest:
257         """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
258         re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
259         the original submit. The row stores exactly the ProcessRequest fields."""
260         return ProcessRequest(
261             video_path=job["video_path"],
262             player_pool=job.get("player_pool"),
263             max_frames=job.get("max_frames"),
264             segmenter=job.get("segmenter"),
265         )
266
267     def _run_claimed_job(job: Dict[str, Any]) -> None:
268         """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
269         its
270         terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
271         Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
272         means
273         the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
274         result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
275         the
276         job self-scheduled and will be re-claimed when `next_poll_at` is due.
277         """
278         job_id = job["id"]
279         req = _job_to_request(job)
280         try:
281             result = _execute_processing(
282                 req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
283             if result.get("video_id"):
284                 db_instance.set_job_video_id(job_id, result["video_id"])

```

```

284         db_instance.mark_job_done(job_id, result)
285     except JobWaiting as w:
286         # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
287         logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
288         db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
289     except Exception as e:
290         logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
291         db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
292
293
294     def _drain_due_jobs() -> int:
295         """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
296         `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
297
298         Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
299         up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
300         a single-worker box keeps making progress with no central timer (the DB is the
clock).
301         Returns the number of jobs that reached a terminal/parked state this tick.
302         """
303         ran = 0
304         while True:
305             job = db_instance.claim_due_job()
306             if job is None:
307                 break
308             ran += 1
309             _run_claimed_job(job)
310         # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
311         # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
312         # on the next submit/drain. Best-effort ? never raises into the worker.
313         _schedule_next_drain_if_waiting()
314         return ran
315
316
317     def _schedule_next_drain_if_waiting() -> None:
318         """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
319         self-scheduled job resumes with no further client submit. The drain itself re-checks
320         due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
321         try:
322             delay = db_instance.seconds_until_next_due()
323         except Exception as e: # pragma: no cover - defensive; never wedge the worker
324             logger.error(f"Could not compute next-due delay: {e}")
325             return
326         if delay is None:
327             return # nothing waiting
328         # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
329         _arm_wake_timer(max(0.0, min(delay, _MAX_DRAIN_WAKE_SEC)))
330
331
332     def _arm_wake_timer(delay_sec: float) -> None:
333         """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
334         scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
335         monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
336         timer = threading.Timer(delay_sec, lambda: _get_executor().submit(_drain_due_jobs))
337         timer.daemon = True

```



```

338         timer.start()
339
340
341     #: A parked job further out than this is re-checked by a capped wake rather than one
long
342     #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
343     _MAX_DRAIN_WAKE_SEC = 60.0
344
345
346     @app.post("/api/process", status_code=202)
347     def process_video(req: ProcessRequest):
348         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
349
350         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
351         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
352         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
353         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
354         """
355         allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
356         try:
357             validate_input_path(req.video_path, allowed_root=allowed_root)
358         except ValueError as e:
359             raise HTTPException(status_code=400, detail=str(e)) from e
360         if not os.path.exists(req.video_path):
361             raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
362         if req.segmenter and not segmenter_registry.get(req.segmenter):
363             available = list(segmenter_registry.list_available().keys())
364             raise HTTPException(
365                 status_code=400,
366                 detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
367             )
368
369         job_id = uuid.uuid4().hex
370         if not db_instance.create_job(job_id, req.video_path, req.segmenter,
371                                     req.player_pool, req.max_frames):
372             raise HTTPException(status_code=500, detail="Failed to persist job record.")
373         _get_executor().submit(_drain_due_jobs)
374         return JSONResponse(
375             status_code=202,
376             content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
377         )
378
379
380     @app.get("/api/jobs/{job_id}")
381     def get_job(job_id: str):
382         """Job state: {status, progress, result, reports}. `status` = execution state
383         (queued|running|done|failed); the pipeline verdict is result["status"]."""
384         job = db_instance.get_job(job_id)
385         if not job:
386             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
387         result = job.get("result")
388         return {
389             "job_id": job["id"],
390             "status": job["status"],
391             "progress": job.get("progress"),
392             "video_id": job.get("video_id"),
393             "error": job.get("error"),
394             "result": result,
395             "reports": (result or {}).get("reports"),
396             "created_at": job.get("created_at"),
397             "started_at": job.get("started_at"),
398             "finished_at": job.get("finished_at"),

```

```

399     }
400
401     # --- Chunked upload (B2, PR-2) -----
402     # Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
403     # proxies cap request bodies at ~100 MB ? HOSTING_PLAN $2), are assembled server-side
404     # under security.upload_dir, then submitted to /api/process by the returned path.
405     # Upload state is in-process by design (this is the single-box alpha): a server restart
406     # aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
407     # arrives with PR-3 identity scoping.
408
409     _uploads: Dict[str, Dict[str, Any]] = {}
410     _uploads_lock = threading.Lock()
411
412
413     class UploadInitRequest(BaseModel):
414         filename: str
415         total_size_bytes: Optional[int] = None
416
417
418     class UploadCompleteRequest(BaseModel):
419         total_parts: int
420
421
422     def _upload_cfg():
423         sec = getattr(global_config, "security", None)
424         upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
425         max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
426         part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
427         exts = [e.lower().lstrip(".") for e in (getattr(sec, "allowed_upload_extensions",
428 None) or ["mp4", "mov"])]
429         return upload_dir, max_bytes, part_bytes, exts
430
431
432     def _abort_upload(upload_id: str) -> None:
433         with _uploads_lock:
434             st = _uploads.pop(upload_id, None)
435             if st:
436                 try:
437                     os.remove(st["tmp_path"])
438                 except OSError:
439                     pass
440
441
442     @app.post("/api/upload/init")
443     def upload_init(req: UploadInitRequest):
444         """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
445         upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
446         try:
447             name = safe_output_filename(req.filename)
448         except ValueError as e:
449             raise HTTPException(status_code=400, detail=str(e)) from e
450         ext = os.path.splitext(name)[1].lower().lstrip(".")
451         if ext not in exts:
452             raise HTTPException(status_code=400,
453 detail=f"Extension '{ext}' not allowed. Allowed:
{sorted(exts)}")
454         if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
455             raise HTTPException(status_code=413,
456 detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB
upload limit.")
457         os.makedirs(upload_dir, exist_ok=True)
458         upload_id = uuid.uuid4().hex
459         tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
460         open(tmp_path, "wb").close()
461         with _uploads_lock:

```

```

461         _uploads[upload_id] = {"filename": name, "tmp_path": tmp_path, "parts": 0,
"bytes": 0}
462     return {"upload_id": upload_id, "max_part_mb": part_bytes // (1024 * 1024),
463           "next_part": f"/api/upload/{upload_id}/part/0"}
464
465
466     @app.put("/api/upload/{upload_id}/part/{index}")
467     async def upload_part(upload_id: str, index: int, request: Request):
468         """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
469         _, max_bytes, part_bytes, _ = _upload_cfg()
470         with _uploads_lock:
471             st = _uploads.get(upload_id)
472             if st is None:
473                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
474             if index != st["parts"]:
475                 raise HTTPException(status_code=409,
476                                   detail=f"Out-of-order part: expected {st['parts']}, got
{index} (parts are sequential)")
477
478             received = 0
479             overflow = None
480             with open(st["tmp_path"], "ab") as fh:
481                 async for chunk in request.stream():
482                     received += len(chunk)
483                     if received > part_bytes:
484                         overflow = f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part
limit."
485                         break
486                     if st["bytes"] + received > max_bytes:
487                         overflow = f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total
limit."
488                         break
489                     fh.write(chunk)
490             if overflow:
491                 _abort_upload(upload_id)
492                 raise HTTPException(status_code=413, detail=overflow + " Upload aborted ? re-
init to retry.")
493
494             with _uploads_lock:
495                 st["parts"] += 1
496                 st["bytes"] += received
497             return {"upload_id": upload_id, "received_parts": st["parts"], "received_bytes":
st["bytes"]}
498
499
500     @app.post("/api/upload/{upload_id}/complete")
501     def upload_complete(upload_id: str, req: UploadCompleteRequest):
502         """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
503         upload_dir, _, _, _ = _upload_cfg()
504         with _uploads_lock:
505             st = _uploads.get(upload_id)
506             if st is None:
507                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
508             if st["parts"] == 0 or st["parts"] != req.total_parts:
509                 raise HTTPException(status_code=409,
510                                   detail=f"Expected {req.total_parts} part(s), received
{st['parts']}.")
511             final_path = os.path.join(upload_dir, st["filename"])
512             if os.path.exists(final_path): # never overwrite an earlier upload with the same
name
513                 stem, ext = os.path.splitext(st["filename"])
514                 final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
515             os.replace(st["tmp_path"], final_path)
516             with _uploads_lock:

```

```

517         _uploads.pop(upload_id, None)
518     video_id = compute_video_id(final_path)
519     return {"path": os.path.abspath(final_path), "video_id": video_id,
520            "size_bytes": os.path.getsize(final_path),
521            "next": "POST /api/process with this path"}
522
523
524 @app.delete("/api/upload/{upload_id}")
525 def upload_abort(upload_id: str):
526     """Abort an in-flight upload and delete its partial file."""
527     with _uploads_lock:
528         known = upload_id in _uploads
529         if not known:
530             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
531         _abort_upload(upload_id)
532     return {"status": "aborted", "upload_id": upload_id}
533
534
535 # --- Highlight download (B3, PR-2) -----
536
537 @app.get("/api/highlights/{video_id}/{filename}")
538 def download_highlight(video_id: str, filename: str):
539     """Stream a compiled highlight reel ? `filename` is the bare name given to
/api/compile
540     (which only writes into output_dir; the browser previously got back an unreachable
541     server-local path)."""
542     try:
543         name = safe_output_filename(filename)
544     except ValueError as e:
545         raise HTTPException(status_code=400, detail=str(e)) from e
546     if not db_instance.get_video(video_id):
547         raise HTTPException(status_code=404, detail="Indexed video record not found.")
548     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
549     or "output"
550     path = os.path.join(output_dir, name)
551     try:
552         path = resolve_under_root(path, output_dir)
553     except ValueError as e:
554         raise HTTPException(status_code=400, detail=str(e)) from e
555     if not os.path.isfile(path):
556         raise HTTPException(status_code=404,
557                             detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
558     media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
559     return FileResponse(path, media_type=media, filename=name)
560
561 @app.post("/api/query")
562 def query_play_segments(req: QueryRequest):
563     filters = {
564         "server": req.server,
565         "receiver": req.receiver,
566         "duration_min": req.duration_min,
567         "event_type": req.event_type
568     }
569     segments = db_instance.query_play_segments(req.video_id, filters)
570     return {"play_segments": segments}
571
572 @app.post("/api/compile")
573 def compile_highlights(req: CompileRequest):
574     video = db_instance.get_video(req.video_id)
575     if not video:
576         raise HTTPException(status_code=404, detail="Indexed video record not found.")
577
578     # Retrieve matching play segments from db

```

```

579     all_segments = db_instance.query_play_segments(req.video_id, {})
580     matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
581
582     if not matched_segments:
583         raise HTTPException(status_code=400, detail="No matching play segments found to
584 stitch.")
585
586     # Reject path traversal / absolute output names (os.path.join would otherwise let an
587 # absolute or '..'-laden output_filename write anywhere on disk).
588     try:
589         out_name = safe_output_filename(req.output_filename)
590     except ValueError as e:
591         raise HTTPException(status_code=400, detail=str(e)) from e
592
593     # The configured shot-count floor (stitching.min_shot_count) applies on the API
594 # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
595 shot_count
596 # metadata are exempt: the floor filters known counts only, so explicitly
597 # requested manual/legacy segments can't silently vanish under the default floor.
598 stitching = getattr(global_config, "stitching", None) or StitchingConfig()
599 kept = []
600 for s in matched_segments:
601     meta = s.get("metadata") or {}
602     sc = meta.get("shot_count") if isinstance(meta, dict) else None
603     try:
604         if sc is not None and int(sc) < stitching.min_shot_count:
605             continue
606     except (TypeError, ValueError):
607         pass # unparseable count -> treat as unknown, keep
608     kept.append(s)
609 if len(kept) < len(matched_segments):
610     logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
611 dropped "
612 f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
613 segments below the floor.")
614 if not kept:
615     raise HTTPException(status_code=400,
616 detail=f"All {len(matched_segments)} matching segments fall
617 below "
618 f"stitching.min_shot_count={stitching.min_shot_count}.")
619
620 # Extract start/end time pairs
621 time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
622
623 # Run compiler with the configured stitching paddings + optional branding watermark
624 output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
625 or "output"
626 compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
627 end_padding=stitching.end_padding,
628 watermark=stitching.watermark)
629 try:
630     out_path = compiler.compile_highlights(video["filepath"], time_pairs, out_name)
631     return {"status": "success", "filepath": out_path}
632 except Exception as e:
633     raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
634 {str(e)}") from e
635
636 # --- CLI Debug Utility & Orchestration ---
637 def run_cli_diagnose_clip(video_path: str, timestamp: float):
638     """CLI debugging utility to examine segment properties around a timestamp."""
639     print("=" * 60)
640     print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
641     print("=" * 60)

```

```

635     cfg = load_config() # legacy wrapper still works, returns dict for now
636
637     # 1. Validation check diagnostics
638     print("[DIAGNOSTIC] Running validation framework...")
639     results = registry.run_all(video_path, cfg)
640     for r in results:
641         status_symbol = "[PASS]" if r.passed else "[FAIL]"
642         print(f" {status_symbol} {r.validator_name}: {r.message}")
643         if r.details:
644             print(f"         Details: {r.details}")
645
646     # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
647     print("-" * 60)
648     print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
649     import cv2
650     cap = cv2.VideoCapture(video_path)
651     if not cap.isOpened():
652         print("[FAIL] OpenCV could not open video.")
653         return
654
655     fps = cap.get(cv2.CAP_PROP_FPS)
656     total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
657
658     start_frame = max(0, int((timestamp - 3.0) * fps))
659     end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
660
661     # Measure frame-by-frame changes in sample region
662     cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
663     prev_gray = None
664     motions = []
665
666     for _ in range(start_frame, end_frame):
667         ret, frame = cap.read()
668         if not ret:
669             break
670         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
671         gray = cv2.GaussianBlur(gray, (21, 21), 0)
672
673         if prev_gray is not None:
674             diff = cv2.absdiff(prev_gray, gray)
675             _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
676             motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
677             motions.append(motion_ratio)
678             prev_gray = gray
679
680     cap.release()
681
682     if motions:
683         avg_motion = sum(motions) / len(motions)
684         # Support both legacy dict (from wrapper) and future direct model
685         if hasattr(cfg, "indexing"):
686             threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
687         else:
688             threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08) # type:
ignore[attr-defined]
689         print(f" Sample Frame Count: {len(motions)}")
690         print(f" Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
691         if avg_motion > threshold:
692             print(" Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
693         else:
694             print(" Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")

```

```

695         else:
696             print(" [ERROR] No visual frames were extracted in the sample range.")
697
698         print("=" * 60)
699
700     def main():
701         parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
702         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
703         parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
704         parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
705         parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
706         parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
707         parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
708         parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
709
710         available_segmenters = list(segmenter_registry.list_available().keys())
711         parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
712         parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
713         parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
714         parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
715
716         args = parser.parse_args()
717
718         if args.setup:
719             # Invoke the diagnostics/bootstrap script (renamed from the old root
720             # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
721             # build system). Resolve its path from this file so `indexer --setup`
722             # works regardless of the caller's cwd.
723             import subprocess
724             doctor_path = os.path.join(
725                 os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
726                 "scripts", "doctor.py",
727             )
728             print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
729             subprocess.run([sys.executable, doctor_path])
730             return
731
732         if args.trim_start is not None and args.trim_end is not None:
733             if not args.video_path:
734                 print("[ERROR] Please provide --video-path to extract a segment.")
735                 return
736
737             # Determine output path
738             out_filename = args.trim_output or "trimmed_segment.mp4"
739             if os.path.isabs(out_filename):
740                 out_path = out_filename
741                 out_dir = os.path.dirname(out_path)
742                 out_name = os.path.basename(out_path)
743             else:
744                 out_dir = args.output_dir
745                 out_path = os.path.join(out_dir, out_filename)
746                 out_name = out_filename
747
748             os.makedirs(out_dir, exist_ok=True)

```

```

749         print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
750
751         # global_config isn't initialized this early; load the typed config so the trim
752         # clip carries the same configured stitching paddings + watermark as a real
753         # highlight (watermark default-on).
754         stitching = load_app_config().stitching
755         compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
756                                 end_padding=stitching.end_padding,
watermark=stitching.watermark)
757         try:
758             res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
759             print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
760         except Exception as e:
761             print(f"[ERROR] Failed to compile trimmed segment: {e}")
762         return
763
764     if args.debug_clip is not None:
765         if not args.video_path:
766             print("[ERROR] Please provide --video-path to debug a specific clip.")
767             return
768         run_cli_diagnose_clip(args.video_path, args.debug_clip)
769         return
770
771     # Initialize Global Config and DB
772     global global_config, db_instance
773     global_config = load_app_config() # returns typed AppConfig
774
775     # The CLI --output-dir overrides the configured output directory. It now lives
776     # on the typed model (OutputConfig.output_dir), so every consumer ? including
777     # /api/compile ? reads the same value instead of a write-only runtime hack.
778     global_config.output.output_dir = args.output_dir
779     os.makedirs(global_config.output.output_dir, exist_ok=True)
780
781     db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
782     db_instance = Database(db_path)
783
784     # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
785     # the executor is in-process. Mark them failed so pollers see a terminal state.
786     swept = db_instance.fail_stale_jobs()
787     if swept:
788         logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
789
790     # CLI processing mode (no web UI needed)
791     if args.video_path and not args.server:
792         print(f"[CLI] Processing video file: {args.video_path}")
793         if not os.path.exists(args.video_path):
794             print(f"[FAIL] Video file not found: {args.video_path}")
795             return
796
797         video_id = compute_video_id(args.video_path)
798         was_reingest = db_instance.get_video(video_id) is not None
799
800         # Validate (with-context variant surfaces ffprobe_meta for the report)
801         print("[CLI] Validating...")
802         val_results, val_context = registry.run_all_with_context(args.video_path,
global_config)
803         failures = [r for r in val_results if not r.passed]
804
805         # Save video status
806         status = "failed" if failures else "processed"
807         db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r

```



```

in val_results])
808
809     seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
810     segmenter_actual = None
811     segmentation_ran = False
812     try:
813         print("\n" + "="*40)
814         print("VALIDATION SUMMARY")
815         print("="*40)
816         for r in val_results:
817             status_symbol = "[PASS]" if r.passed else "[FAIL]"
818             print(f"{status_symbol} {r.validator_name}: {r.message}")
819         print("="*40 + "\n")
820
821         if failures:
822             print("[FAIL] Video failed checks. Indexing halted.")
823             return
824
825         # Index
826         segmenter_cls = segmenter_registry.get(seg_name)
827         if not segmenter_cls:
828             print(f"[FAIL] Segmenter '{seg_name}' not found.")
829             return
830
831         print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
832         segmenter_actual = seg_name
833         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
834         segmenter = segmenter_cls(db_instance, global_config)
835         result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
836         segmentation_ran = True
837
838         if isinstance(result, Failure):
839             print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
840             print("[FAIL] Indexing halted for video.")
841             segments = []
842         else:
843             segments = result.value if hasattr(result, "value") else result
844             print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
845             # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
846             _finalize_reingest(video_id, segments, play_wm, cand_wm)
847         finally:
848             # Always emit the per-video observability report. Never raises.
849             paths = emit_indexer_report(
850                 db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
851                 val_results=val_results, val_context=val_context,
852                 segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
853                 was_reingest=was_reingest, segmentation_ran=segmentation_ran,
854             )
855             if paths:
856                 print(f"[REPORT] Technical report : {paths.get('technical_md')}")
857                 print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
858                 print(f"[REPORT] Machine JSON      : {paths.get('json')}")
859             return
860
861         # Start Local Server Mode
862         if args.server or not args.video_path:
863             print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
864             # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the

```

```

865         # in-process wake-timer for any parked 'waiting' job died with the old process,
866         so
867         # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
868         _get_executor().submit(_drain_due_jobs)
869         uvicorn.run(app, host="127.0.0.1", port=args.port)
870
871 if __name__ == "__main__":
872     main()

```

6. assistant (2026-06-18T07:54:46.500Z)

Now let me read the database.py file and the test files for both.

7. user (2026-06-18T07:54:47.853Z)

```

1     import sqlite3
2     import json
3     import logging
4     from contextlib import contextmanager
5     from typing import List, Dict, Any, Optional, Tuple
6
7     logger = logging.getLogger(__name__)
8
9     class Database:
10         def __init__(self, db_path: str):
11             self.db_path = db_path
12             self._init_db()
13
14         @contextmanager
15         def _connect(self):
16             """Transaction scope that also CLOSES the connection.
17
18             ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
19             never closes, so every call leaked its connection to GC (lingering file
20             handles on the WAL sidecars under Windows). Internal methods use this.
21             """
22             conn = self._get_connection()
23             try:
24                 with conn:
25                     yield conn
26             finally:
27                 conn.close()
28
29         def _get_connection(self):
30             # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
31             # adds concurrent writers (a swallowed 'database is locked' silently drops
32             segments).
33             conn = sqlite3.connect(self.db_path, timeout=30.0)
34             conn.row_factory = sqlite3.Row
35             # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
36             conn.execute("PRAGMA foreign_keys = ON")
37             # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
38             conn.execute("PRAGMA busy_timeout = 30000")
39             try:
40                 conn.execute("PRAGMA journal_mode = WAL")
41             except sqlite3.OperationalError:
42                 pass # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
43             return conn
44
45         @staticmethod
46         def _add_column_idempotent(cursor: sqlite3.Cursor, ddl: str) -> None:
47             """Apply a bare ``ALTER TABLE ... ADD COLUMN`` migration step re-runnably.
48
49             A DDL ``ALTER`` auto-commits independently of the surrounding ``with conn:``

```

```

49         transaction, so an interrupted v3/v4 block can leave the column added while
50         ``user_version`` stays un-bumped ? and the next open would re-run the same
51         ``ALTER`` and raise ``OperationalError: duplicate column name``, bricking every
52         future ``Database(...)`` construction (``_init_db`` runs in ``__init__``).
53     Swallow
54     ONLY that specific error so the ladder stays crash-safe, matching the re-
55     runnable
56     ``CREATE ... IF NOT EXISTS`` v1/v2/v5 blocks.
57     """
58     try:
59         cursor.execute(ddl)
60     except sqlite3.OperationalError as exc:
61         if "duplicate column name" not in str(exc).lower():
62             raise
63
64     def _init_db(self):
65         """Initializes tables for videos, play_segments, and events."""
66         with self._connect() as conn:
67             cursor = conn.cursor()
68
69             # Videos Table
70             cursor.execute("""
71                 CREATE TABLE IF NOT EXISTS videos (
72                     id TEXT PRIMARY KEY,
73                     filepath TEXT NOT NULL,
74                     processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
75                     status TEXT NOT NULL,
76                     validation_results TEXT
77                 )
78             """)
79
80             # Play Segments Table (Extensible metadata JSON)
81             cursor.execute("""
82                 CREATE TABLE IF NOT EXISTS play_segments (
83                     id INTEGER PRIMARY KEY AUTOINCREMENT,
84                     video_id TEXT NOT NULL,
85                     start_time REAL NOT NULL,
86                     end_time REAL NOT NULL,
87                     duration REAL NOT NULL,
88                     server TEXT,
89                     receiver TEXT,
90                     metadata TEXT,
91                     FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
92                 )
93             """)
94
95             # Events Table
96             cursor.execute("""
97                 CREATE TABLE IF NOT EXISTS events (
98                     id INTEGER PRIMARY KEY AUTOINCREMENT,
99                     segment_id INTEGER NOT NULL,
100                     timestamp REAL NOT NULL,
101                     type TEXT NOT NULL,
102                     player TEXT,
103                     details TEXT,
104                     FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE
105                     CASCADE
106                 )
107             """)
108
109             # Versioned migrations live here. PRAGMA user_version is the schema
110             # contract ? bump it for every change and add an ordered `if version < N`
111             # block below. Tests assert the expected version, so accidental drift fails
112
113             version = cursor.execute("PRAGMA user_version").fetchone()[0]

```

```

110
111         if version < 1:
112             # v1: raw candidate detections (pre-confirmation), detector-agnostic.
113             # Common fields are columns; detector-specific metrics go in `stats`
114             JSON,
115             # so a new segmenter reuses this table by setting its own
116             `detector`/`stats`.
117             cursor.execute("""
118                 CREATE TABLE IF NOT EXISTS candidate_segments (
119                     id INTEGER PRIMARY KEY AUTOINCREMENT,
120                     video_id TEXT NOT NULL,
121                     detector TEXT NOT NULL,
122                     chunk_index INTEGER,
123                     start_time REAL NOT NULL,
124                     end_time REAL NOT NULL,
125                     duration REAL NOT NULL,
126                     confidence REAL,
127                     stats TEXT,
128                     detection_params TEXT,
129                     confirmed_segment_id INTEGER,
130                     human_label TEXT,
131                     notes TEXT,
132                     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
133                     FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
134                     FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id)
135                     ON DELETE SET NULL
136                 )
137             """)
138             cursor.execute("""
139                 CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
140                 ON candidate_segments(video_id, detector)
141             """)
142             cursor.execute("PRAGMA user_version = 1")
143
144         if version < 2:
145             # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per
146             submitted
147             # /api/process request. `status` is the EXECUTION state of the job
148             # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
149             # that failed validation) lives inside `result` JSON as
150             result["status"].
151             # `result` is the full sync-era response dict (incl. report paths), so
152             the
153             # observability report is the job's artifact, per the plan.
154             cursor.execute("""
155                 CREATE TABLE IF NOT EXISTS jobs (
156                     id TEXT PRIMARY KEY,
157                     video_path TEXT NOT NULL,
158                     video_id TEXT,
159                     segmenter TEXT,
160                     player_pool TEXT,
161                     max_frames INTEGER,
162                     status TEXT NOT NULL DEFAULT 'queued',
163                     progress TEXT,
164                     error TEXT,
165                     result TEXT,
166                     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
167                     started_at TIMESTAMP,
168                     finished_at TIMESTAMP
169                 )
170             """)
171             cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON
172             jobs(status)")
173             cursor.execute("PRAGMA user_version = 2")

```

```

168         if version < 3:
169             # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2).
170             # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting'
171             # job is due
172             # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ?
173             # any worker
174             # re-claims it via `claim_due_job` when due ? with NO master node: the
175             # table IS
176             # the coordinator. ALTER (not recreate) so existing v2 job rows survive;
177             # ADD COLUMNS are applied idempotently so an interrupted upgrade can re-
178             # run them.
179             self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN policy
180             TEXT")
181             self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN
182             next_poll_at TIMESTAMP")
183             cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
184             next_poll_at)")
185             cursor.execute("PRAGMA user_version = 3")
186
187         if version < 4:
188             # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a
189             # NULLABLE
190             # `user_id` to videos + candidate_segments so a future per-user / multi-
191             # tenant
192             # path can scope rows to an owner. **NULL = local install = byte-
193             # identical to
194             # today** ? every existing row stays NULL and every existing query
195             # (which never
196             # survives;
197             # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows
198             # survive;
199             # the ADD COLUMNS are applied idempotently so an interrupted upgrade can
200             # re-run them.
201             # Additive ONLY: no served-behavior change rides on this slice.
202             self._add_column_idempotent(cursor, "ALTER TABLE videos ADD COLUMN
203             user_id TEXT")
204             self._add_column_idempotent(cursor, "ALTER TABLE candidate_segments ADD
205             COLUMN user_id TEXT")
206             # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast
207             # path free
208             # of index churn while making the eventual per-user lookups cheap.
209             cursor.execute(
210                 "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
211                 "WHERE user_id IS NOT NULL")
212             cursor.execute(
213                 "CREATE INDEX IF NOT EXISTS idx_candidates_user "
214                 "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
215             cursor.execute("PRAGMA user_version = 4")
216
217         if version < 5:
218             # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per
219             # (user_id,
220             # version): the resolved per-user `fusion_config` overlay + an INLINE
221             # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
222             # evidence
223             # and provenance that earned it. `is_active` pins the one row the
224             # serving bridge
225             # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`).
226             # This slice
227             # only persists/loads it ? NOTHING reads it on the served path yet.
228             #
229             # user_id                ? owner of the model (NULL allowed = the local
230             # install)
231             # version                ? monotonic per-user model version

```

```

210         # baseline_version      ? the shared baseline this was fit against
(upgrade switch)
211         # feature_schema_hash    ? fingerprint of the feature set; MINOR-vs-
MAJOR re-fit gate
212         # fusion_config          ? JSON per-user FusionConfig overlay (cue_flags
/ thresholds)
213         # classifier_json        ? JSON LogisticRegression.to_dict() (None =
config-only model)
214         # promotion_evidence     ? JSON per_user_gate PromotionDecision (why it
was promoted)
215         # n_videos_at_fit        ? held-out-user corpus size when fit (min_n
trust floor input)
216         # is_active              ? the one resolved row for this user (partial-
unique below)
217         cursor.execute("""
218             CREATE TABLE IF NOT EXISTS user_models (
219                 id INTEGER PRIMARY KEY AUTOINCREMENT,
220                 user_id TEXT,
221                 version INTEGER NOT NULL DEFAULT 1,
222                 baseline_version TEXT,
223                 feature_schema_hash TEXT,
224                 fusion_config TEXT,
225                 classifier_json TEXT,
226                 promotion_evidence TEXT,
227                 n_videos_at_fit INTEGER NOT NULL DEFAULT 0,
228                 is_active INTEGER NOT NULL DEFAULT 0,
229                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
230             )
231         """)
232         # One model version per user (NULLs compare distinct in SQLite, so the
local
233         # install can still carry multiple versioned rows; the active-row guard
below
234         # is what enforces a single served model).
235         cursor.execute("""
236             CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
237             ON user_models(user_id, version)
238         """)
239         # At most ONE active model per user: a partial-unique index over the
active rows.
240         # (SQLite treats NULL user_id rows as distinct, so the local install's
single
241         # active row is still guarded ? there is one local user.)
242         cursor.execute("""
243             CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_one_active
244             ON user_models(user_id) WHERE is_active = 1
245         """)
246         cursor.execute("PRAGMA user_version = 5")
247         # future: if version < 6: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 6
248
249         conn.commit()
250
251     def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
252         """Register or update a video row WITHOUT touching its child segments.
253
254         Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
255         with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
256         That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
257         before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
258         row in place; clearing segments is now an explicit, deliberate operation

```

```

259         (clear_video_data / prune_segments).
260         """
261         try:
262             with self._connect() as conn:
263                 conn.cursor().execute(
264                     "INSERT INTO videos (id, filepath, status, validation_results)
VALUES (?, ?, ?, ?) "
265                     "ON CONFLICT(id) DO UPDATE SET "
266                     "filepath=excluded.filepath, status=excluded.status, "
267                     "validation_results=excluded.validation_results",
268                     (video_id, filepath, status, json.dumps(validation_results))
269                 )
270                 conn.commit()
271             return True
272         except Exception as e:
273             logger.error(f"Failed to add video: {e}")
274             return False
275
276     def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
277         """Return (max play_segment id, max candidate_segment id) for a video (0 if
278         none).
279         Captured before a (re)segmentation so the run's new rows (id > watermark) can be
280         told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
281         """
282         with self._connect() as conn:
283             cur = conn.cursor()
284             p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
video_id = ?",
285                             (video_id,)).fetchone()[0]
286             c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
video_id = ?",
287                             (video_id,)).fetchone()[0]
288             return int(p), int(c)
289
290     def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
291                       play_after: Optional[int] = None, cand_upto: Optional[int] =
None,
292                       cand_after: Optional[int] = None) -> None:
293         """Delete play/candidate segments by id range (events cascade from
play_segments).
294
295         Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
296         successful run (`*_upto` = the pre-run watermark), or drop the NEW partial
rows
297         and keep the OLD ones when the run failed/produced nothing (`*_after`).
298         """
299         with self._connect() as conn:
300             cur = conn.cursor()
301             if play_upto is not None:
302                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
303             if play_after is not None:
304                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
305             if cand_upto is not None:
306                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
?", (video_id, cand_upto))
307             if cand_after is not None:
308                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
?", (video_id, cand_after))
309             conn.commit()
310
311     def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]

```

```

= None) -> Optional[int]:
312         try:
313             with self._connect() as conn:
314                 cursor = conn.cursor()
315                 cursor.execute(
316                     "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
317                     (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {})))
318                 )
319                 conn.commit()
320                 return cursor.lastrowid
321         except Exception as e:
322             logger.error(f"Failed to add play segment: {e}")
323             return None
324
325     def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
326         try:
327             with self._connect() as conn:
328                 conn.cursor().execute(
329                     "INSERT INTO events (segment_id, timestamp, type, player, details)
VALUES (?, ?, ?, ?, ?)",
330                     (segment_id, timestamp, event_type, player, json.dumps(details or
{})))
331                 )
332                 conn.commit()
333                 return True
334         except Exception as e:
335             logger.error(f"Failed to add event: {e}")
336             return False
337
338     def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
339                               chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
340                               stats: Optional[Dict[str, Any]] = None,
341                               detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
342         """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
343         detector-specific dicts stored as JSON. Returns the new candidate id, or
344         None."""
345         try:
346             with self._connect() as conn:
347                 cursor = conn.cursor()
348                 cursor.execute(
349                     """INSERT INTO candidate_segments
350                     (video_id, detector, chunk_index, start_time, end_time, duration,
351                     confidence, stats, detection_params)
352                     VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
353                     (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
354                     confidence, json.dumps(stats or {}), json.dumps(detection_params or
{})))
355                 )
356                 conn.commit()
357                 return cursor.lastrowid
358         except Exception as e:
359             logger.error(f"Failed to add candidate segment: {e}")
360             return None
361
362     def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
363         """Record that a candidate detection was confirmed as a given play_segment."""
364         try:
365             with self._connect() as conn:

```



```

365         conn.cursor().execute(
366             "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id =
367             ?",
368             (segment_id, candidate_id)
369         )
370         conn.commit()
371         return True
372     except Exception as e:
373         logger.error(f"Failed to link candidate {candidate_id} to segment
374         {segment_id}: {e}")
375         return False
376
377     def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
378                                only_unlabeled: bool = False) -> List[Dict[str, Any]]:
379         """Fetch candidate detections for the annotation / fine-tuning workflow.
380         `stats` and `detection_params` are deserialized from JSON."""
381         query = "SELECT * FROM candidate_segments WHERE video_id = ?"
382         params: List[Any] = [video_id]
383
384         if detector:
385             query += " AND detector = ?"
386             params.append(detector)
387         if only_unlabeled:
388             query += " AND human_label IS NULL"
389
390         query += " ORDER BY start_time"
391
392         candidates = []
393         with self._connect() as conn:
394             rows = conn.cursor().execute(query, params).fetchall()
395             for row in rows:
396                 d = dict(row)
397                 d["stats"] = json.loads(d["stats"] or "{}")
398                 d["detection_params"] = json.loads(d["detection_params"] or "{}")
399                 candidates.append(d)
400         return candidates
401
402     def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
403         with self._connect() as conn:
404             row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
405             (video_id,)).fetchone()
406             if row:
407                 d = dict(row)
408                 d["validation_results"] = json.loads(d["validation_results"] or "[]")
409                 return d
410             return None
411
412     def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
413     List[Dict[str, Any]]:
414         """
415         Dynamically queries play segments based on filters:
416         - duration_min: float
417         - server: string
418         - receiver: string
419         - event_type: string (e.g. smash, foul)
420         """
421         query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
422         params = [video_id]
423
424         if filters.get("duration_min"):
425             query += " AND r.duration >= ?"
426             params.append(filters["duration_min"])
427
428         if filters.get("server"):
429             query += " AND r.server = ?"

```

```

426         params.append(filters["server"])
427
428     if filters.get("receiver"):
429         query += " AND r.receiver = ?"
430         params.append(filters["receiver"])
431
432     if filters.get("event_type"):
433         # Check if there is an event of a specific type inside this segment
434         query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
435         params.append(filters["event_type"])
436
437     segments_list = []
438     with self._connect() as conn:
439         cursor = conn.cursor()
440         rows = cursor.execute(query, params).fetchall()
441
442         for row in rows:
443             r_dict = dict(row)
444             r_id = r_dict["id"]
445             r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
446
447             # Fetch associated events
448             event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
449             r_dict["events"] = []
450             for er in event_rows:
451                 e_dict = dict(er)
452                 e_dict["details"] = json.loads(e_dict["details"] or "{}")
453                 r_dict["events"].append(e_dict)
454
455             segments_list.append(r_dict)
456
457     return segments_list
458
459     def clear_video_data(self, video_id: str):
460         with self._connect() as conn:
461             conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
462             conn.commit()
463
464     # --- Async job store (DEFERRED_HARDENING_PLAN #16) -----
465     # Writers follow the repo convention (swallow + logger.error, return bool); the
466     # worker logs loudly on a failed transition so a job can't silently wedge.
467
468     def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
469                   player_pool: Optional[List[str]] = None,
470                   max_frames: Optional[int] = None,
471                   policy: Optional[Dict[str, Any]] = None) -> bool:
472         """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
473         (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy."""
474         try:
475             with self._connect() as conn:
476                 conn.cursor().execute(
477                     "INSERT INTO jobs (id, video_path, segmenter, player_pool,
max_frames, status, policy) "
478                     "VALUES (?, ?, ?, ?, ?, 'queued', ?)",
479                     (job_id, video_path, segmenter,
480                      json.dumps(player_pool) if player_pool is not None else None,
481                      max_frames, json.dumps(policy) if policy is not None else None)
482                 )
483                 conn.commit()
484             return True
485         except Exception as e:
486             logger.error(f"Failed to create job {job_id}: {e}")

```

```

487         return False
488
489     def mark_job_running(self, job_id: str) -> bool:
490         try:
491             with self._connect() as conn:
492                 conn.cursor().execute(
493                     "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
494                     "progress='starting' WHERE id = ?", (job_id,))
495                 conn.commit()
496             return True
497         except Exception as e:
498             logger.error(f"Failed to mark job {job_id} running: {e}")
499             return False
500
501     def set_job_progress(self, job_id: str, progress: str) -> bool:
502         try:
503             with self._connect() as conn:
504                 conn.cursor().execute(
505                     "UPDATE jobs SET progress=? WHERE id = ?", (progress, job_id))
506                 conn.commit()
507             return True
508         except Exception as e:
509             logger.error(f"Failed to set progress for job {job_id}: {e}")
510             return False
511
512     def set_job_video_id(self, job_id: str, video_id: str) -> bool:
513         try:
514             with self._connect() as conn:
515                 conn.cursor().execute(
516                     "UPDATE jobs SET video_id=? WHERE id = ?", (video_id, job_id))
517                 conn.commit()
518             return True
519         except Exception as e:
520             logger.error(f"Failed to set video_id for job {job_id}: {e}")
521             return False
522
523     def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
524         """Job EXECUTED to completion. The pipeline's own verdict (success/failed
525         validation) is inside `result` ? job status 'done' means 'the worker
526         finished'."""
527         try:
528             with self._connect() as conn:
529                 conn.cursor().execute(
530                     "UPDATE jobs SET status='done', progress='finished', result=?, "
531                     "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
532                     (json.dumps(result), job_id))
533                 conn.commit()
534             return True
535         except Exception as e:
536             logger.error(f"Failed to mark job {job_id} done: {e}")
537             return False
538
539     def mark_job_failed(self, job_id: str, error: str) -> bool:
540         """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
541         try:
542             with self._connect() as conn:
543                 conn.cursor().execute(
544                     "UPDATE jobs SET status='failed', error=?, "
545                     "finished_at=CURRENT_TIMESTAMP WHERE id = ?", (error, job_id))
546                 conn.commit()
547             return True
548         except Exception as e:
549             logger.error(f"Failed to mark job {job_id} failed: {e}")
550             return False

```

```

551     def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
552         """Fetch one job; `result`/`player_pool` JSON deserialized."""
553         with self._connect() as conn:
554             row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
555 (job_id,)).fetchone()
556             if not row:
557                 return None
558             d = dict(row)
559             d["result"] = json.loads(d["result"]) if d["result"] else None
560             d["player_pool"] = json.loads(d["player_pool"]) if d["player_pool"] else
561             None
562             d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
563             return d
564
565     # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2)
566     -----
567     def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
568         """Attach/replace a job's JSON ExecutionPolicy."""
569         try:
570             with self._connect() as conn:
571                 conn.cursor().execute(
572                     "UPDATE jobs SET policy=? WHERE id = ?",
573                     (json.dumps(policy) if policy is not None else None, job_id))
574                 conn.commit()
575             return True
576         except Exception as e:
577             logger.error(f"Failed to set policy for job {job_id}: {e}")
578             return False
579
580     def set_job_waiting(self, job_id: str, delay_sec: float, progress: Optional[str] =
581     None) -> bool:
582         """Self-schedule: park a job in 'waiting' until ``delay_sec`` from now,
583         releasing the
584         worker. `claim_due_job` re-picks it when due ? cooperative, no master.
585         ``next_poll_at`` is
586         computed in SQLite's own UTC format so it compares directly to
587         CURRENT_TIMESTAMP."""
588         try:
589             with self._connect() as conn:
590                 conn.cursor().execute(
591                     "UPDATE jobs SET status='waiting', "
592                     "next_poll_at=datetime('now', ?), "
593                     "progress=COALESCE(?, progress) WHERE id = ?",
594                     (f"+{max(0, int(delay_sec))} seconds", progress, job_id))
595                 conn.commit()
596             return True
597         except Exception as e:
598             logger.error(f"Failed to set job {job_id} waiting: {e}")
599             return False
600
601     def seconds_until_next_due(self) -> Optional[float]:
602         """Seconds until the soonest parked ('waiting') job becomes due, so the worker
603         can
604         schedule a wake-up (EXECUTION_POLICY.md P3 self-scheduling). Returns 0.0 if one
605         is
606         already due, or None if nothing is waiting. A NULL ``next_poll_at`` ? due now
607         (0.0)."""
608         try:
609             with self._connect() as conn:
610                 row = conn.cursor().execute(
611                     "SELECT MIN(CASE WHEN next_poll_at IS NULL THEN 0 ELSE "
612                     " (julianday(next_poll_at) - julianday('now')) * 86400.0 END) AS
613                     secs "
614                     "FROM jobs WHERE status='waiting')."
615                 if row is None or row["secs"] is None:

```

```

605         return None
606         return max(0.0, float(row["secs"]))
607     except Exception as e:
608         logger.error(f"Failed to compute next-due delay: {e}")
609         return None
610
611     def claim_due_job(self) -> Optional[Dict[str, Any]]:
612         """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
613         ``next_poll_at`` is due ?
614         flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
615         never
616         double-claim (the DB row-claim is the only coordination needed; no master).
617         Earliest-due
618         first. Returns the claimed job dict, or None if nothing is runnable."""
619         try:
620             with self._connect() as conn:
621                 cur = conn.cursor()
622                 row = cur.execute(
623                     "SELECT id, status FROM jobs WHERE status='queued' "
624                     "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
625                     CURRENT_TIMESTAMP)) "
626                     "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT
627                     1").fetchone()
628             if not row:
629                 return None
630             cur.execute(
631                 "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP "
632                 "WHERE id=? AND status=?", (row["id"], row["status"]))
633             conn.commit()
634             if cur.rowcount != 1:
635                 return None # lost the race to a concurrent
636             worker
637
638             return self.get_job(row["id"])
639         except Exception as e:
640             logger.error(f"Failed to claim a due job: {e}")
641             return None
642
643     def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
644         """Crash recovery: any job still queued/running from a previous process is dead
645         (the executor is in-process). Mark failed so clients see a terminal state, not a
646         hang. Called once at startup. Returns the number of jobs swept."""
647         try:
648             with self._connect() as conn:
649                 cur = conn.cursor()
650                 cur.execute(
651                     "UPDATE jobs SET status='failed', error=?,
652                     finished_at=CURRENT_TIMESTAMP "
653                     "WHERE status IN ('queued', 'running')", (reason,))
654                 conn.commit()
655                 return cur.rowcount
656         except Exception as e:
657             logger.error(f"Failed to sweep stale jobs: {e}")
658             return 0
659
660     # --- Per-user model store (PERSONALIZATION_PLAN.md §2, v5) -----
661     # Groundwork ONLY: these persist/load the per-user model row. NOTHING on the served
662     # pipeline reads them yet ? the serving bridge (backend/pipeline/personalization/
663     # serving.py) resolves them, and wiring that bridge into the production decision
664     # seam
665     # (`fusion_hybrid._select_for_handoff`) is the NEXT, owner-gated PR. NULL user_id =
666     # the local install. `fusion_config` / `classifier_json` / `promotion_evidence` are
667     # JSON-serialised dicts; the rest are scalar columns.
668
669     def upsert_user_model(self, user_id: Optional[str], *, version: int = 1,
670                           baseline_version: Optional[str] = None,

```

```

662         feature_schema_hash: Optional[str] = None,
663         fusion_config: Optional[Dict[str, Any]] = None,
664         classifier_json: Optional[Dict[str, Any]] = None,
665         promotion_evidence: Optional[Dict[str, Any]] = None,
666         n_videos_at_fit: int = 0,
667         is_active: bool = False) -> Optional[int]:
668     """Insert or replace ONE per-user model row keyed by ``(user_id, version)``.
669
670     Idempotent on that key (UPSERT, not INSERT-OR-REPLACE: REPLACE would re-allocate
671     the
672     rowid). When ``is_active`` is True this row becomes the user's single active
673     model ?
674     any previously-active row for the SAME user is first cleared, so the
675     ``idx_user_models_one_active`` partial-unique invariant always holds (one active
676     model
677     per user). Returns the row id, or None on failure.
678     """
679     try:
680         with self._connect() as conn:
681             cur = conn.cursor()
682             if is_active:
683                 # Demote the current active row first (NULL user_id = local: IS NULL
684                 match).
685                 if user_id is None:
686                     cur.execute(
687                         "UPDATE user_models SET is_active = 0 "
688                         "WHERE user_id IS NULL AND is_active = 1 AND version != ?",
689                         (version,))
690                 else:
691                     cur.execute(
692                         "UPDATE user_models SET is_active = 0 "
693                         "WHERE user_id = ? AND is_active = 1 AND version != ?",
694                         (user_id, version))
695             cur.execute(
696                 """INSERT INTO user_models
697                 (user_id, version, baseline_version, feature_schema_hash,
698                 fusion_config, classifier_json, promotion_evidence,
699                 n_videos_at_fit, is_active)
700                 VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
701                 ON CONFLICT(user_id, version) DO UPDATE SET
702                 baseline_version=excluded.baseline_version,
703                 feature_schema_hash=excluded.feature_schema_hash,
704                 fusion_config=excluded.fusion_config,
705                 classifier_json=excluded.classifier_json,
706                 promotion_evidence=excluded.promotion_evidence,
707                 n_videos_at_fit=excluded.n_videos_at_fit,
708                 is_active=excluded.is_active""",
709                 (user_id, version, baseline_version, feature_schema_hash,
710                 json.dumps(fusion_config) if fusion_config is not None else None,
711                 json.dumps(classifier_json) if classifier_json is not None else
712                 None,
713                 json.dumps(promotion_evidence) if promotion_evidence is not None
714                 else None,
715                 n_videos_at_fit, 1 if is_active else 0))
716             conn.commit()
717             rid = cur.execute(
718                 "SELECT id FROM user_models WHERE "
719                 + ("user_id IS NULL" if user_id is None else "user_id = ?")
720                 + " AND version = ?",
721                 ((version,) if user_id is None else (user_id, version))).fetchone()
722             return int(rid[0]) if rid else None
723     except Exception as e:
724         logger.error(f"Failed to upsert user_model (user={user_id}, v={version}):
725         {e}")
726     return None

```

```

720
721     def get_active_user_model(self, user_id: Optional[str]) -> Optional[Dict[str, Any]]:
722         """Return the single active per-user model row for ``user_id`` (NULL = local
install),
723         with ``fusion_config`` / ``classifier_json`` / ``promotion_evidence`` JSON
deserialised,
724         or None if the user has no active model. Does not touch the served pipeline."""
725         with self._connect() as conn:
726             if user_id is None:
727                 row = conn.cursor().execute(
728                     "SELECT * FROM user_models WHERE user_id IS NULL AND is_active =
1").fetchone()
729             else:
730                 row = conn.cursor().execute(
731                     "SELECT * FROM user_models WHERE user_id = ? AND is_active = 1",
732                     (user_id,)).fetchone()
733             if not row:
734                 return None
735             d = dict(row)
736             for col in ("fusion_config", "classifier_json", "promotion_evidence"):
737                 d[col] = json.loads(d[col]) if d[col] else None
738             return d
739

```

8. assistant (2026-06-18T07:54:50.433Z)

Now let me look at the test files for both, and check how main.py's helpers are imported/used by tests.

9. user (2026-06-18T07:54:51.160Z)

```

1     import sqlite3
2     import pytest
3     from backend.infrastructure.database import Database
4
5     def test_database_init(tmp_path):
6         db_path = tmp_path / "test.db"
7         Database(str(db_path)) # instantiation creates the schema
8
9         conn = sqlite3.connect(str(db_path))
10        cursor = conn.cursor()
11        cursor.execute("SELECT name FROM sqlite_master WHERE type='table'")
12        tables = [r[0] for r in cursor.fetchall()]
13        assert "videos" in tables
14        assert "play_segments" in tables
15        conn.close()
16
17    def test_database_video(tmp_path):
18        db = Database(str(tmp_path / "test.db"))
19
20        vid_added = db.add_video("vid1", "vid1.mp4", "pending", [{"val": 1}])
21        assert vid_added is True
22
23        video = db.get_video("vid1")
24        assert video["id"] == "vid1"
25        assert video["filepath"] == "vid1.mp4"
26        assert video["status"] == "pending"
27        assert video["validation_results"] == [{"val": 1}]
28
29        db.clear_video_data("vid1")
30        assert db.get_video("vid1") is None
31
32    def test_database_play_segment(tmp_path):
33        db = Database(str(tmp_path / "test.db"))
34        db.add_video("vid1", "vid1.mp4", "pending", [])

```

```

35
36     sid = db.add_play_segment("vid1", 10.0, 20.0, "PlayerA", "PlayerB", {"foo": "bar"})
37     assert sid is not None
38
39     db.add_event(sid, 15.0, "smash", "PlayerA", {"speed": 300})
40
41     segments = db.query_play_segments("vid1", {"server": "PlayerA"})
42     assert len(segments) == 1
43     assert segments[0]["id"] == sid
44     assert segments[0]["start_time"] == 10.0
45     assert segments[0]["server"] == "PlayerA"
46     assert segments[0]["metadata"]["foo"] == "bar"
47
48     assert len(segments[0]["events"]) == 1
49     assert segments[0]["events"][0]["type"] == "smash"
50     assert segments[0]["events"][0]["details"]["speed"] == 300
51
52
53     # --- candidate_segments schema + behavior -----
54
55     # The schema contract: any future change to candidate_segments columns must
56     # consciously update this set, otherwise the test below fails (regression guard).
57     EXPECTED_CANDIDATE_COLUMNS = {
58         "id", "video_id", "detector", "chunk_index", "start_time", "end_time",
59         "duration", "confidence", "stats", "detection_params",
60         "confirmed_segment_id", "human_label", "notes", "created_at",
61         "user_id", # v4: personalization groundwork (nullable; NULL = local install)
62     }
63
64
65     def test_candidate_schema_contract(tmp_path):
66         Database(str(tmp_path / "test.db")) # instantiation creates the schema
67         conn = sqlite3.connect(str(tmp_path / "test.db"))
68         cols = conn.execute("PRAGMA table_info(candidate_segments)").fetchall()
69         conn.close()
70
71         names = {c[1] for c in cols}
72         assert names == EXPECTED_CANDIDATE_COLUMNS
73
74         # NOT NULL / PK flags on the load-bearing columns (cid, name, type, notnull, dflt,
75         pk)
76         by_name = {c[1]: c for c in cols}
77         assert by_name["id"][5] == 1 # primary key
78         for col in ("video_id", "detector", "start_time", "end_time", "duration"):
79             assert by_name[col][3] == 1, f"{col} should be NOT NULL"
80
81     def test_schema_version_is_current(tmp_path):
82         db_path = tmp_path / "test.db"
83         Database(str(db_path))
84         conn = sqlite3.connect(str(db_path))
85         version = conn.execute("PRAGMA user_version").fetchone()[0]
86         conn.close()
87         # v5 = per-user model store (PERSONALIZATION_PLAN.md §2). Bump with each migration.
88         assert version == 5
89
90
91     def test_init_db_idempotent_and_upgrades_old_db(tmp_path):
92         db_path = tmp_path / "legacy.db"
93         # Simulate a pre-v1 database that only has the old tables and no version.
94         conn = sqlite3.connect(str(db_path))
95         conn.execute("CREATE TABLE videos (id TEXT PRIMARY KEY, filepath TEXT, status
96         TEXT)")
97         conn.commit()
98         conn.close()

```



```

98
99     # Opening with the new Database should add candidate_segments without error...
100     Database(str(db_path))
101     # ...and running init again must be a no-op (idempotent).
102     Database(str(db_path))
103
104     conn = sqlite3.connect(str(db_path))
105     tables = [r[0] for r in conn.execute(
106         "SELECT name FROM sqlite_master WHERE type='table'").fetchall()]
107     version = conn.execute("PRAGMA user_version").fetchone()[0]
108     conn.close()
109     assert "candidate_segments" in tables
110     assert "jobs" in tables # v2 (async job model, #16)
111     assert "user_models" in tables # v5 (per-user model store)
112     assert version == 5 # v5 = per-user model store (PERSONALIZATION_PLAN.md §2)
113
114
115 def test_candidate_round_trip(tmp_path):
116     db = Database(str(tmp_path / "test.db"))
117     db.add_video("vid1", "vid1.mp4", "processed", [])
118
119     cid = db.add_candidate_segment(
120         "vid1", "yolo_hybrid", 10.0, 25.0, chunk_index=0, confidence=0.42,
121         stats={"peak_velocity": 0.8, "track_id_count": 2},
122         detection_params={"frame_skip": 3},
123     )
124     assert cid is not None
125
126     cands = db.query_candidate_segments("vid1")
127     assert len(cands) == 1
128     c = cands[0]
129     assert c["detector"] == "yolo_hybrid"
130     assert c["start_time"] == 10.0
131     assert c["duration"] == 15.0
132     assert c["confidence"] == 0.42
133     assert c["stats"]["peak_velocity"] == 0.8
134     assert c["detection_params"]["frame_skip"] == 3
135     assert c["confirmed_segment_id"] is None
136
137     # detector filter
138     assert db.query_candidate_segments("vid1", detector="motion") == []
139     assert len(db.query_candidate_segments("vid1", detector="yolo_hybrid")) == 1
140
141
142 def test_candidate_link_and_unlabeled_filter(tmp_path):
143     db = Database(str(tmp_path / "test.db"))
144     db.add_video("vid1", "vid1.mp4", "processed", [])
145
146     cid = db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)
147     sid = db.add_play_segment("vid1", 1.0, 4.0)
148
149     assert db.link_candidate_to_segment(cid, sid) is True
150     assert db.query_candidate_segments("vid1")[0]["confirmed_segment_id"] == sid
151
152     # only_unlabeled returns it while human_label is NULL
153     assert len(db.query_candidate_segments("vid1", only_unlabeled=True)) == 1
154     with db._get_connection() as conn:
155         conn.execute("UPDATE candidate_segments SET human_label = ? WHERE id = ?",
156             ("true_rally", cid))
157         conn.commit()
158     assert db.query_candidate_segments("vid1", only_unlabeled=True) == []
159
160
161 def test_candidate_cascade_on_video_delete(tmp_path):
162     db = Database(str(tmp_path / "test.db"))

```

```

163     db.add_video("vid1", "vid1.mp4", "processed", [])
164     db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)
165
166     db.clear_video_data("vid1")
167     assert db.query_candidate_segments("vid1") == []
168
169
170     # --- v4: personalization groundwork (nullable user_id; NULL = local = unchanged) -----
171
172     def _table_cols(db_path, table):
173         conn = sqlite3.connect(str(db_path))
174         cols = {c[1] for c in conn.execute(f"PRAGMA table_info({table})").fetchall()}
175         conn.close()
176         return cols
177
178
179     def test_v4_user_id_columns_present_and_nullable(tmp_path):
180         db_path = tmp_path / "test.db"
181         Database(str(db_path))
182         assert "user_id" in _table_cols(db_path, "videos")
183         assert "user_id" in _table_cols(db_path, "candidate_segments")
184
185         # The column is NULLABLE: PRAGMA table_info notnull flag must be 0.
186         conn = sqlite3.connect(str(db_path))
187         for table in ("videos", "candidate_segments"):
188             by_name = {c[1]: c for c in conn.execute(f"PRAGMA
table_info({table})").fetchall()}
189             assert by_name["user_id"][3] == 0, f"{table}.user_id must be nullable"
190             conn.close()
191
192
193     def test_v4_existing_writers_leave_user_id_null(tmp_path):
194         """NULL user_id back-compat: add_video / add_candidate_segment never set user_id, so
every
195         row a current writer produces is local (NULL) ? byte-identical to pre-v4
behaviour."""
196         db_path = tmp_path / "test.db"
197         db = Database(str(db_path))
198         db.add_video("vid1", "vid1.mp4", "processed", [])
199         db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)
200
201         conn = sqlite3.connect(str(db_path))
202         vu = conn.execute("SELECT user_id FROM videos WHERE id = 'vid1']").fetchone()[0]
203         cu = conn.execute("SELECT user_id FROM candidate_segments WHERE video_id =
'vid1']").fetchone()[0]
204         conn.close()
205         assert vu is None
206         assert cu is None
207         # Existing readers are unaffected (no user_id filter in the query path).
208         assert len(db.query_candidate_segments("vid1")) == 1
209
210
211     def test_migration_v3_db_upgrades_to_v5_preserving_rows(tmp_path):
212         """A real v3 DB (jobs + candidates already populated) upgrades cleanly to v5: the
new
213         columns/tables appear, user_id back-fills to NULL, and prior rows survive the
ALTERs."""
214         db_path = tmp_path / "legacy_v3.db"
215         db = Database(str(db_path)) # fresh DB is born at the current version; simulate a
v3 state
216         # Seed pre-upgrade data, then force the version back to 3 and re-open to run v4+v5.
217         db.add_video("vidA", "a.mp4", "processed", [{"v": 1}])
218         db.add_candidate_segment("vidA", "wasb_hybrid", 2.0, 9.0, stats={"net_crossings":
3})
219         conn = sqlite3.connect(str(db_path))

```

```

220 conn.execute("DROP INDEX IF EXISTS idx_videos_user")
221 conn.execute("DROP INDEX IF EXISTS idx_candidates_user")
222 conn.execute("DROP TABLE IF EXISTS user_models")
223 # Recreate v3-shaped tables WITHOUT user_id by rebuilding (simplest: just rewind
version
224 # and drop the v4/v5 artifacts that a true v3 DB wouldn't have).
225 conn.execute("ALTER TABLE videos DROP COLUMN user_id")
226 conn.execute("ALTER TABLE candidate_segments DROP COLUMN user_id")
227 conn.execute("PRAGMA user_version = 3")
228 conn.commit()
229 conn.close()
230 assert "user_id" not in _table_cols(db_path, "videos")
231
232 # Re-open: the migration ladder must carry the v3 DB up to v5 without data loss.
233 db2 = Database(str(db_path))
234 conn = sqlite3.connect(str(db_path))
235 assert conn.execute("PRAGMA user_version").fetchone()[0] == 5
236 assert "user_models" in {r[0] for r in conn.execute(
237     "SELECT name FROM sqlite_master WHERE type='table'").fetchall()}
238 conn.close()
239 assert "user_id" in _table_cols(db_path, "videos")
240 assert "user_id" in _table_cols(db_path, "candidate_segments")
241
242 v = db2.get_video("vidA")
243 assert v is not None and v["validation_results"] == [{"v": 1}]
244 cands = db2.query_candidate_segments("vidA")
245 assert len(cands) == 1 and cands[0]["stats"]["net_crossings"] == 3
246
247
248 def test_migration_idempotent_on_interrupted_upgrade(tmp_path):
249     """An INTERRUPTED v3/v4 upgrade must not brick the DB. A DDL ``ALTER`` auto-commits
250     on its own, so a crash after the ADD COLUMNs but before the ``PRAGMA user_version``
251     bump leaves the columns present while the version is stale ? re-opening re-runs the
252     bare ALTERs. The ladder must tolerate that (re-runnable, like the v1/v2/v5 blocks)
253     instead of raising ``OperationalError: duplicate column name``."""
254     db_path = tmp_path / "interrupted.db"
255     Database(str(db_path)) # fully migrate to the current version (v3/v4 columns all
present)
256
257     # Simulate the interruption: the v3/v4 ADD COLUMNs already landed and auto-
committed,
258     # but the process died before the version bump, so user_version is stuck at 2.
259     conn = sqlite3.connect(str(db_path))
260     conn.execute("DROP TABLE IF EXISTS user_models") # a true v2 DB lacks the
v5 table
261     conn.execute("PRAGMA user_version = 2")
262     conn.commit()
263     conn.close()
264
265     # Re-opening re-runs v3 (jobs.policy/next_poll_at) and v4 (videos/candidate_segments
266     # .user_id) over columns that ALREADY exist ? must NOT raise, and must reach v5.
267     Database(str(db_path))
268     conn = sqlite3.connect(str(db_path))
269     assert conn.execute("PRAGMA user_version").fetchone()[0] == 5
270     conn.close()
271     # Columns/tables are all present after the clean re-apply.
272     assert "policy" in _table_cols(db_path, "jobs")
273     assert "user_id" in _table_cols(db_path, "videos")
274     assert "user_id" in _table_cols(db_path, "candidate_segments")
275     conn = sqlite3.connect(str(db_path))
276     assert "user_models" in {r[0] for r in conn.execute(
277         "SELECT name FROM sqlite_master WHERE type='table'").fetchall()}
278     conn.close()
279
280

```

```

281 def test_add_column_idempotent_swallows_only_duplicate(tmp_path):
282     """The idempotent ADD COLUMN helper swallows ONLY 'duplicate column name'. Any other
283     OperationalError (e.g. a genuinely broken migration against a missing table) must
still
284     propagate ? the swallow is narrow, not a blanket except."""
285     db_path = tmp_path / "helper.db"
286     Database(str(db_path)) # current schema; jobs.policy already
exists
287     conn = sqlite3.connect(str(db_path))
288     cur = conn.cursor()
289     # Re-adding an existing column is tolerated (the crash-safety path).
290     Database._add_column_idempotent(cur, "ALTER TABLE jobs ADD COLUMN policy TEXT")
291     # An unrelated OperationalError (no such table) is NOT masked.
292     with pytest.raises(sqlite3.OperationalError):
293         Database._add_column_idempotent(cur, "ALTER TABLE nope ADD COLUMN x TEXT")
294     conn.close()
295
296
297 # --- v5: per-user model store CRUD -----
298
299 def test_user_models_schema_contract(tmp_path):
300     db_path = tmp_path / "test.db"
301     Database(str(db_path))
302     cols = _table_cols(db_path, "user_models")
303     assert cols == {
304         "id", "user_id", "version", "baseline_version", "feature_schema_hash",
305         "fusion_config", "classifier_json", "promotion_evidence",
306         "n_videos_at_fit", "is_active", "created_at",
307     }
308
309
310 def test_upsert_and_get_active_user_model_round_trip(tmp_path):
311     db = Database(str(tmp_path / "test.db"))
312     rid = db.upsert_user_model(
313         "user-1", version=1, baseline_version="base-2026-06",
314         feature_schema_hash="abc123",
315         fusion_config={"min_crossings": 3, "cue_flags": {"person": True}},
316         classifier_json={"model": "logreg", "w": [0.1, 0.2], "b": 0.0},
317         promotion_evidence={"promote": True, "n": 6},
318         n_videos_at_fit=6, is_active=True,
319     )
320     assert rid is not None
321
322     m = db.get_active_user_model("user-1")
323     assert m is not None
324     assert m["user_id"] == "user-1"
325     assert m["version"] == 1
326     assert m["baseline_version"] == "base-2026-06"
327     assert m["feature_schema_hash"] == "abc123"
328     assert m["fusion_config"] == {"min_crossings": 3, "cue_flags": {"person": True}}
329     assert m["classifier_json"]["w"] == [0.1, 0.2]
330     assert m["promotion_evidence"]["promote"] is True
331     assert m["n_videos_at_fit"] == 6
332     assert m["is_active"] == 1
333
334
335 def test_get_active_user_model_none_when_absent_or_inactive(tmp_path):
336     db = Database(str(tmp_path / "test.db"))
337     assert db.get_active_user_model("nobody") is None
338
339     # An inactive model is not returned as active.
340     db.upsert_user_model("user-2", version=1, is_active=False)
341     assert db.get_active_user_model("user-2") is None
342
343

```

```

344 def test_upsert_is_idempotent_on_user_version(tmp_path):
345     db = Database(str(tmp_path / "test.db"))
346     rid1 = db.upsert_user_model("user-3", version=1, n_videos_at_fit=4, is_active=True)
347     rid2 = db.upsert_user_model(
348         "user-3", version=1, n_videos_at_fit=7, is_active=True,
349         fusion_config={"min_crossings": 5},
350     )
351     # Same (user, version) ? same row updated in place, not a duplicate.
352     assert rid1 == rid2
353     m = db.get_active_user_model("user-3")
354     assert m["n_videos_at_fit"] == 7
355     assert m["fusion_config"] == {"min_crossings": 5}
356
357     conn = sqlite3.connect(str(tmp_path / "test.db"))
358     count = conn.execute(
359         "SELECT COUNT(*) FROM user_models WHERE user_id = 'user-3'").fetchone()[0]
360     conn.close()
361     assert count == 1
362
363
364 def test_activating_new_version_demotes_prior_active(tmp_path):
365     db = Database(str(tmp_path / "test.db"))
366     db.upsert_user_model("user-4", version=1, n_videos_at_fit=5, is_active=True)
367     db.upsert_user_model("user-4", version=2, n_videos_at_fit=9, is_active=True)
368
369     active = db.get_active_user_model("user-4")
370     assert active["version"] == 2 # the newer model is the single active one
371
372     # Exactly one active row for the user (the partial-unique invariant holds).
373     conn = sqlite3.connect(str(tmp_path / "test.db"))
374     n_active = conn.execute(
375         "SELECT COUNT(*) FROM user_models WHERE user_id = 'user-4' AND is_active =
376         1").fetchone()[0]
377     conn.close()
378     assert n_active == 1
379
380 def test_null_user_id_local_install_round_trip(tmp_path):
381     """NULL user_id = the local install. upsert/get must treat IS NULL correctly (not
382     '=NULL')."""
383     db = Database(str(tmp_path / "test.db"))
384     db.upsert_user_model(None, version=1, fusion_config={"min_crossings": 2},
385         n_videos_at_fit=5, is_active=True)
386     m = db.get_active_user_model(None)
387     assert m is not None
388     assert m["user_id"] is None
389     assert m["fusion_config"] == {"min_crossings": 2}
390     # A named user does NOT see the local row.
391     assert db.get_active_user_model("user-x") is None
392
393 def test_user_model_json_columns_default_none(tmp_path):
394     """A config-only model (no classifier) stores NULL classifier_json and deserialises
395     to None."""
396     db = Database(str(tmp_path / "test.db"))
397     db.upsert_user_model("user-5", version=1, fusion_config={"min_crossings": 4},
398         n_videos_at_fit=5, is_active=True)
399     m = db.get_active_user_model("user-5")
400     assert m["classifier_json"] is None
401     assert m["promotion_evidence"] is None
402     assert m["fusion_config"] == {"min_crossings": 4}

```

Now let me look at the API endpoint tests, async job tests, and check what tests exist that exercise main.py's logic (especially the upload helpers, CLI diagnose, and the finalize/reingest helpers).

11. user (2026-06-18T07:54:56.763Z)

```
323 tests/test_api_endpoints.py
352 tests/test_async_jobs.py
 85 tests/test_job_self_scheduling.py
292 tests/test_job_worker_loop.py
 81 tests/test_reingest_safety.py
 49 tests/test_path_safety.py
 38 tests/test_reliability_hardening.py
1220 total
```

12. user (2026-06-18T07:54:59.990Z)

```
1      """TestClient coverage for the FastAPI seam (previously zero ? the compiler SyntaxError
2      that broke `import backend.main` shipped green because nothing imported it)."""
3      import os
4      import subprocess
5      from unittest.mock import patch, MagicMock
6
7      import pytest
8
9      pytest.importorskip("httpx") # TestClient needs httpx; skip cleanly where absent
10     from fastapi.testclient import TestClient
11
12     import backend.main as m
13     from backend.infrastructure.database import Database
14     from backend.config.models import AppConfig
15     from backend.pipeline.validators.base import ValidationResult
16
17
18     class _InlineExecutor:
19         """submit() runs the fn synchronously ? async jobs complete before the response
20         is inspected (full lifecycle coverage lives in test_async_jobs.py)."""
21
22         def submit(self, fn, *args, **kwargs):
23             fn(*args, **kwargs)
24
25
26     @pytest.fixture
27     def env(tmp_path, monkeypatch):
28         db = Database(str(tmp_path / "t.db"))
29         cfg = AppConfig()
30         monkeypatch.setattr(m, "db_instance", db)
31         monkeypatch.setattr(m, "global_config", cfg)
32         monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
33         client = TestClient(m.app)
34         return client, db, tmp_path, monkeypatch
35
36
37     def _video(tmp_path, name="m.mp4"):
38         p = tmp_path / name
39         p.write_bytes(b"not-a-real-video-but-hashable")
40         return p
41
42
43     def _pass():
44         return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]
45
46
47     def _fail():
48         return [ValidationResult(validator_name="blur", passed=False, message="blurry",
```

```

details={}])
49
50
51     class _FakeSeg:
52         def __init__(self, db, cfg):
53             self.db = db
54
55         def process_video(self, video_id, video_path, *a, **k):
56             sid = self.db.add_play_segment(video_id, 0.0, 5.0)
57             return [{"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0,
"metadata": {}}]
58
59
60     def test_api_config_ok(env):
61         client = env[0]
62         r = client.get("/api/config")
63         assert r.status_code == 200
64         assert r.json()["sport"] == "badminton"
65
66
67     def test_process_success_with_mocked_segmenter(env):
68         """#16: submit returns 202 + job_id; the (inline) job carries the old response."""
69         client, db, tmp_path, mp = env
70         vid_file = _video(tmp_path)
71         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
72         mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
73         r = client.post("/api/process", json={"video_path": str(vid_file), "segmenter":
"x"})
74         assert r.status_code == 202
75         job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
76         assert job["status"] == "done"
77         assert job["result"]["status"] == "success"
78         assert len(job["result"]["play_segments"]) == 1
79
80
81     def test_process_validation_failure_preserves_prior_segments(env):
82         client, db, tmp_path, mp = env
83         vid_file = _video(tmp_path)
84         vid = m.compute_video_id(str(vid_file))
85         db.add_video(vid, str(vid_file), "processed", [])
86         db.add_play_segment(vid, 1.0, 2.0) # prior good result
87         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_fail(), {}))
88         r = client.post("/api/process", json={"video_path": str(vid_file)})
89         assert r.status_code == 202
90         job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
91         assert job["status"] == "done" # worker finished; the PIPELINE verdict failed
92         assert job["result"]["status"] == "failed"
93         # PR-C: a failed re-validation must NOT cascade-delete prior segments.
94         assert len(db.query_play_segments(vid, {})) == 1
95
96
97     def test_process_rejects_null_byte_path(env):
98         client = env[0]
99         r = client.post("/api/process", json={"video_path": "a\x00b.mp4"})
100         assert r.status_code == 400
101
102
103     # =====
104     # Item 4 positive tests (output layout honors configured output_dir)
105     # =====
106
107     def test_process_uses_configured_output_dir_not_literal_output(env, tmp_path,
monkeypatch):
108         """Positive test for Item 4: when output.output_dir overridden, artifacts/paths
109         use the configured root; no leakage to literal 'output/' for the video_id.

```

```

110
111     This would have caught re-introduction of hard-coded os.path.join("output", ...)
112     (see main.py:548/619 and similar in compiler/detectors). Uses the existing
113     env fixture + tmp_path for isolation, matching reingest safety + compiler patterns.
114     """
115     client, db, tpath, mp = env
116     vid_file = _video(tpath)
117     vid = m.compute_video_id(str(vid_file))
118
119     custom_out = str(tmp_path / "custom_layout_out")
120     cfg = AppConfig()
121     cfg.output.output_dir = custom_out
122     mp.setattr(m, "global_config", cfg)
123
124     # ensure the custom dir is "used" (makedirs would happen in real main for CLI)
125     os.makedirs(custom_out, exist_ok=True)
126
127     mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
128     mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
129
130     r = client.post("/api/process", json={"video_path": str(vid_file)})
131     assert r.status_code == 202
132
133     job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
134     assert job["status"] == "done"
135     # positive: the response/result paths should reflect configured
136     assert "result" in job
137     # In real flow compiler/main would join under custom_out; here we assert config
reached
138     assert cfg.output.output_dir == custom_out
139     # use vid and test real no-leakage to literal output/<vid> (core of Item 4)
140     literal = os.path.join("output", vid)
141     assert not os.path.exists(literal), f"should not leak to literal output/ for vid
{vid}"
142     # (full FS/golden verification in owner protocol per §5/§7)
143
144
145     def test_compiler_and_reingest_safety_honor_output_dir_override(env, tmp_path,
monkeypatch):
146         """Positive for Item 4: layout check combined with reingest safety pattern.
147         Override output_dir; failed run must not pollute literal 'output/'; success uses
configured.
148         Mirrors plan to use test_api_endpoints + reingest safety + compiler.
149         """
150         client, db, tpath, mp = env
151         vid_file = _video(tpath)
152         vid = m.compute_video_id(str(vid_file))
153
154         custom_out = str(tmp_path / "reingest_custom_out")
155         cfg = AppConfig()
156         cfg.output.output_dir = custom_out
157         mp.setattr(m, "global_config", cfg)
158         os.makedirs(custom_out, exist_ok=True)
159
160         # first "run" that fails validation (reingest safety)
161         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_fail(), {}))
162         r = client.post("/api/process", json={"video_path": str(vid_file)})
163         assert r.status_code == 202
164         job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
165         assert job["result"]["status"] == "failed"
166         # prior good would be preserved (existing test); here assert config dir was the
context
167         assert cfg.output.output_dir == custom_out
168         # use vid and test no leakage in the fail path too
169         literal = os.path.join("output", vid)

```



```

170         assert not os.path.exists(literal), f"should not leak to literal output/ for vid
{vid}"
171
172         # success path also uses configured (no literal output/ introduced)
173         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
174         r2 = client.post("/api/process", json={"video_path": str(vid_file)})
175         assert r2.status_code == 202
176         assert cfg.output.output_dir == custom_out # still honored
177         # use vid and test no leakage in success path
178         literal = os.path.join("output", vid)
179         assert not os.path.exists(literal), f"should not leak to literal output/ for vid
{vid}"
180
181
182
183     def test_query_returns_segments(env):
184         client, db, tmp_path, mp = env
185         db.add_video("vid", "/x.mp4", "processed", [])
186         db.add_play_segment("vid", 1.0, 4.0)
187         r = client.post("/api/query", json={"video_id": "vid"})
188         assert r.status_code == 200
189         assert len(r.json()["play_segments"]) == 1
190
191
192     def test_compile_rejects_traversal_output_filename(env):
193         client, db, tmp_path, mp = env
194         db.add_video("vid", str(video(tmp_path)), "processed", [])
195         sid = db.add_play_segment("vid", 1.0, 4.0)
196         r = client.post("/api/compile", json={
197             "video_id": "vid", "segment_ids": [sid], "output_filename": "../evil.mp4"})
198         assert r.status_code == 400
199
200
201     def test_compile_success_with_mocked_compiler(env):
202         client, db, tmp_path, mp = env
203         db.add_video("vid", str(video(tmp_path)), "processed", [])
204         sid = db.add_play_segment("vid", 1.0, 4.0)
205
206         class _FakeCompiler:
207             def __init__(self, out_dir, watermark=None, **kwargs):
208                 pass
209
210             def compile_highlights(self, raw, pairs, name):
211                 return f"output/{name}"
212
213         mp.setattr(m, "VideoCompiler", _FakeCompiler)
214         r = client.post("/api/compile", json={
215             "video_id": "vid", "segment_ids": [sid], "output_filename": "out.mp4"})
216         assert r.status_code == 200
217         assert r.json()["filepath"] == "output/out.mp4"
218
219
220     def test_compile_enforces_config_min_shot_count(env):
221         """stitching.min_shot_count applies on the API path (parity with
222         run_process_and_stitch.py): below-floor segments are dropped; segments
223         WITHOUT shot_count metadata are exempt (explicitly requested ids must not
224         silently vanish under the default floor)."""
225         client, db, tmp_path, mp = env
226         m.global_config.stitching.min_shot_count = 8
227         db.add_video("vid", str(video(tmp_path)), "processed", [])
228         s_low = db.add_play_segment("vid", 1.0, 4.0, None, None, {"shot_count": 5})
229         s_high = db.add_play_segment("vid", 10.0, 14.0, None, None, {"shot_count": 12})
230         s_unknown = db.add_play_segment("vid", 20.0, 24.0) # no shot_count metadata
231
232         pairs_seen = []

```

```

233
234 class _FakeCompiler:
235     def __init__(self, out_dir, **kwargs):
236         pass
237
238     def compile_highlights(self, raw, pairs, name):
239         pairs_seen.extend(pairs)
240         return f"output/{name}"
241
242 mp.setattr(m, "VideoCompiler", _FakeCompiler)
243 r = client.post("/api/compile", json={
244     "video_id": "vid", "segment_ids": [s_low, s_high, s_unknown],
245     "output_filename": "reel.mp4"})
246 assert r.status_code == 200
247 assert (10.0, 14.0) in pairs_seen and (20.0, 24.0) in pairs_seen
248 assert (1.0, 4.0) not in pairs_seen
249
250
251 def test_compile_400_when_all_below_min_shot_count(env):
252     client, db, tmp_path, mp = env
253     m.global_config.stitching.min_shot_count = 8
254     db.add_video("vid", str(_video(tmp_path)), "processed", [])
255     sid = db.add_play_segment("vid", 1.0, 4.0, None, None, {"shot_count": 3})
256     r = client.post("/api/compile", json={
257         "video_id": "vid", "segment_ids": [sid], "output_filename": "reel.mp4"})
258     assert r.status_code == 400
259     assert "min_shot_count" in r.json()["detail"]
260
261
262 def test_compile_uses_configured_stitching_paddings(env):
263     """config.json stitching.{begin,end}_padding must reach the ffmpeg trim boundaries.
264     Regression for the 2026-06-10 audit finding (d): /api/compile constructed
265     VideoCompiler without the paddings, so reels always used the constructor
266     defaults (0.5/1.0) and the config values were silently ignored."""
267     client, db, tmp_path, mp = env
268     cfg = m.global_config
269     cfg.output.output_dir = str(tmp_path / "out")
270     cfg.stitching.begin_padding = 2.0
271     cfg.stitching.end_padding = 3.0
272     cfg.stitching.watermark.enabled = False # keep the trim command minimal
273     db.add_video("vid", str(_video(tmp_path)), "processed", [])
274     sid = db.add_play_segment("vid", 10.0, 14.0)
275
276     calls = []
277
278     def fake_run(cmd, *a, **k):
279         calls.append(cmd)
280         if cmd[0] == "ffprobe":
281             return MagicMock(stdout="100.0\n", returncode=0)
282         out = cmd[-1]
283         if isinstance(out, str) and out.endswith(".mp4"):
284             open(out, "wb").write(b"clip")
285         return MagicMock(returncode=0, stdout=b"", stderr=b"")
286
287     with patch.object(subprocess, "run", side_effect=fake_run):
288         r = client.post("/api/compile", json={
289             "video_id": "vid", "segment_ids": [sid], "output_filename": "reel.mp4"})
290         assert r.status_code == 200
291
292         trim = next(c for c in calls if c[0] == "ffmpeg" and "-ss" in c)
293         # 10.0 - begin_padding 2.0 and 14.0 + end_padding 3.0; the unwired constructor
294         # defaults (0.5/1.0) would have produced 9.5/15.0 here.
295         assert trim[trim.index("-ss") + 1] == "8.0"
296         assert trim[trim.index("-to") + 1] == "17.0"
297

```

```

298
299     def test_parity_job_worker_vs_direct_cli_path(env, tmp_path, monkeypatch):
300         """Item 5 positive: same ProcessRequest through job worker path and direct CLI-style
path
301         produce byte-identical DB rows, reports, final play_segments (modulo timestamps).
302         Exercises Failure and empty-run branches.
303         """
304         client, db, tpath, mp = env
305         vid_file = _video(tpath)
306         vid = m.compute_video_id(str(vid_file))
307         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
308         mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
309         # job worker path
310         rj = client.post("/api/process", json={"video_path": str(vid_file), "segmenter":
"x"})
311         jobj = client.get(f"/api/jobs/{rj.json()['job_id']}").json()
312         # direct CLI-style path (using same processing setup as main CLI; inline executor
makes outcomes identical in test harness)
313         rd = client.post("/api/process", json={"video_path": str(vid_file), "segmenter":
"x"})
314         jobd = client.get(f"/api/jobs/{rd.json()['job_id']}").json()
315         # byte-identical key outcomes
316         assert jobj["result"]["status"] == jobd["result"]["status"]
317         assert len(db.query_play_segments(vid, {})) == len(db.query_play_segments(vid, {}))
318         # for failure
319         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_fail(), {}))
320         rf = client.post("/api/process", json={"video_path": str(vid_file)})
321         jobf = client.get(f"/api/jobs/{rf.json()['job_id']}").json()
322         assert jobf["result"]["status"] == "failed"
323         # empty-run similar (would preserve prior via reingest safety in both paths)
324

```

13. assistant (2026-06-18T07:55:03.144Z)

Now let me check the upload tests and the async job tests for coverage of the upload helpers and job-worker functions.

14. user (2026-06-18T07:55:06.404Z)

```

test_generate_highlights.py
test_upload_download.py
---grep upload in tests---
tests/test_config_wasb.py
tests/test_gemini.py
tests/test_path_safety.py
tests/test_provider_gemini.py
tests/test_tracknet_hybrid.py
tests/test_upload_download.py
---grep run_cli_diagnose / debug_clip / _upload_cfg---
tests/test_async_jobs.py
tests/test_job_worker_loop.py

```

15. assistant (2026-06-18T07:55:08.945Z)

The upload test file exists. Let me read it and the async jobs test to understand coverage of the job-worker seam and upload helpers.

16. user (2026-06-18T07:55:09.299Z)

```

1     """Chunked upload (B2) + highlight download (B3) ? PR-2 of the UI alpha plan.
2
3     Contract under test:
4     - POST /api/upload/init ? {upload_id}; extension allowlist + size caps enforced
5     - PUT /api/upload/{id}/part/{n} ? strictly sequential parts; per-part/total caps abort

```

```

(413)
6      - POST /api/upload/{id}/complete ? assembled file, MD5 == compute_video_id, no overwrite
7      - DELETE /api/upload/{id} ? abort removes partial state
8      - GET /api/highlights/{video_id}/{filename} ? streams from output_dir; traversal-proof
9      """
10     import os
11
12     import pytest
13
14     pytest.importorskip("httpx") # TestClient needs httpx; skip cleanly where absent
15     from fastapi.testclient import TestClient
16
17     import backend.main as m
18     from backend.infrastructure.database import Database
19     from backend.config.models import AppConfig
20     from backend.utils.hashing import compute_video_id
21
22
23     @pytest.fixture
24     def env(tmp_path, monkeypatch):
25         db = Database(str(tmp_path / "t.db"))
26         cfg = AppConfig()
27         cfg.output.output_dir = str(tmp_path / "out")
28         cfg.security.upload_dir = str(tmp_path / "up")
29         os.makedirs(cfg.output.output_dir, exist_ok=True)
30         monkeypatch.setattr(m, "db_instance", db)
31         monkeypatch.setattr(m, "global_config", cfg)
32         monkeypatch.setattr(m, "_uploads", {}) # isolate upload state per test
33         client = TestClient(m.app)
34         return client, db, tmp_path, cfg
35
36
37     def _init(client, filename="match.mp4", total=None):
38         body = {"filename": filename}
39         if total is not None:
40             body["total_size_bytes"] = total
41         return client.post("/api/upload/init", json=body)
42
43
44     # --- init -----
45
46     def test_init_rejects_bad_extension(env):
47         client = env[0]
48         r = _init(client, "malware.exe")
49         assert r.status_code == 400
50         assert "not allowed" in r.json()["detail"]
51
52
53     def test_init_rejects_traversal_filename(env):
54         client = env[0]
55         assert _init(client, "../evil.mp4").status_code == 400
56         assert _init(client, "a/b.mp4").status_code == 400
57
58
59     def test_init_rejects_oversize_declared_total(env):
60         client, db, tmp_path, cfg = env
61         cfg.security.max_upload_mb = 1
62         r = _init(client, total=2 * 1024 * 1024)
63         assert r.status_code == 413
64
65
66     # --- full lifecycle -----
67
68     def test_roundtrip_assembles_parts_and_matches_md5(env):
69         client, db, tmp_path, cfg = env

```

```

70     up = _init(client).json()
71     parts = [b"a" * 1000, b"b" * 2000, b"c" * 500]
72     for i, blob in enumerate(parts):
73         r = client.put(f"/api/upload/{up['upload_id']}/part/{i}", content=blob)
74         assert r.status_code == 200
75         assert r.json()["received_parts"] == i + 1
76     done = client.post(f"/api/upload/{up['upload_id']}/complete", json={"total_parts":
3})
77     assert done.status_code == 200
78     body = done.json()
79     assert body["size_bytes"] == 3500
80     with open(body["path"], "rb") as fh:
81         assert fh.read() == b"".join(parts)
82     assert body["video_id"] == compute_video_id(body["path"])
83     # partial file cleaned up, registry entry gone
84     assert not [f for f in os.listdir(cfg.security.upload_dir) if
f.startswith(".partial-")]
85     assert client.put(f"/api/upload/{up['upload_id']}/part/3", content=b"x").status_code
== 404
86
87
88     def test_same_filename_twice_does_not_overwrite(env):
89         client = env[0]
90         paths = []
91         for content in (b"first", b"second"):
92             up = _init(client).json()
93             client.put(f"/api/upload/{up['upload_id']}/part/0", content=content)
94             done = client.post(f"/api/upload/{up['upload_id']}/complete",
json={"total_parts": 1})
95             paths.append(done.json()["path"])
96             assert paths[0] != paths[1]
97             with open(paths[0], "rb") as fh:
98                 assert fh.read() == b"first" # first upload untouched
99             with open(paths[1], "rb") as fh:
100                 assert fh.read() == b"second"
101
102
103     # --- sequencing & caps -----
104
105     def test_out_of_order_part_is_409(env):
106         client = env[0]
107         up = _init(client).json()
108         assert client.put(f"/api/upload/{up['upload_id']}/part/1", content=b"x").status_code
== 409
109
110
111     def test_unknown_upload_id_is_404(env):
112         client = env[0]
113         assert client.put("/api/upload/nope/part/0", content=b"x").status_code == 404
114         assert client.post("/api/upload/nope/complete", json={"total_parts": 1}).status_code
== 404
115         assert client.delete("/api/upload/nope").status_code == 404
116
117
118     def test_complete_with_wrong_part_count_is_409(env):
119         client = env[0]
120         up = _init(client).json()
121         client.put(f"/api/upload/{up['upload_id']}/part/0", content=b"x")
122         assert client.post(
123             f"/api/upload/{up['upload_id']}/complete", json={"total_parts": 2}).status_code
== 409
124         # zero-part complete is also a 409
125         up2 = _init(client).json()
126         assert client.post(
127             f"/api/upload/{up2['upload_id']}/complete", json={"total_parts": 0}).status_code

```

```

== 409
128
129
130 def test_oversize_part_aborts_upload(env):
131     client, db, tmp_path, cfg = env
132     cfg.security.max_part_mb = 1
133     up = _init(client).json()
134     r = client.put(f"/api/upload/{up['upload_id']}/part/0",
135                   content=b"x" * (1024 * 1024 + 10))
136     assert r.status_code == 413
137     # aborted: id unknown afterwards, no partials on disk
138     assert client.put(f"/api/upload/{up['upload_id']}/part/0", content=b"x").status_code
== 404
139     assert not [f for f in os.listdir(cfg.security.upload_dir) if
140 f.startswith(".partial-")]
141
142 def test_total_cap_across_parts_aborts_upload(env):
143     client, db, tmp_path, cfg = env
144     cfg.security.max_upload_mb = 1
145     up = _init(client).json()
146     ok = client.put(f"/api/upload/{up['upload_id']}/part/0", content=b"x" * (700 *
147 1024))
148     assert ok.status_code == 200
149     r = client.put(f"/api/upload/{up['upload_id']}/part/1", content=b"x" * (700 * 1024))
150     assert r.status_code == 413
151
152 def test_abort_endpoint_removes_state(env):
153     client, db, tmp_path, cfg = env
154     up = _init(client).json()
155     client.put(f"/api/upload/{up['upload_id']}/part/0", content=b"x")
156     assert client.delete(f"/api/upload/{up['upload_id']}").status_code == 200
157     assert client.put(f"/api/upload/{up['upload_id']}/part/1", content=b"x").status_code
== 404
158     assert not [f for f in os.listdir(cfg.security.upload_dir) if
159 f.startswith(".partial-")]
160
161 # --- download (B3)
-----
162
163 def test_download_unknown_video_404(env):
164     client = env[0]
165     assert client.get("/api/highlights/nope/reel.mp4").status_code == 404
166
167
168 def test_download_traversal_filename_rejected(env):
169     client, db, tmp_path, cfg = env
170     db.add_video("vid", "/x.mp4", "processed", [])
171     r = client.get("/api/highlights/vid/..%2Fevil.mp4")
172     assert r.status_code in (400, 404) # 400 from safe-name; 404 if the router
normalizes
173     r2 = client.get("/api/highlights/vid/a%5Cb.mp4") # backslash separator
174     assert r2.status_code == 400
175
176
177 def test_download_missing_file_404(env):
178     client, db, tmp_path, cfg = env
179     db.add_video("vid", "/x.mp4", "processed", [])
180     assert client.get("/api/highlights/vid/reel.mp4").status_code == 404
181
182
183 def test_download_streams_compiled_file(env):
184     client, db, tmp_path, cfg = env

```

```

185         db.add_video("vid", "/x.mp4", "processed", [])
186         blob = b"\x00\x00fake-mp4-bytes" * 100
187         with open(os.path.join(cfg.output.output_dir, "reel.mp4"), "wb") as fh:
188             fh.write(blob)
189         r = client.get("/api/highlights/vid/reel.mp4")
190         assert r.status_code == 200
191         assert r.content == blob
192         assert r.headers["content-type"].startswith("video/mp4")
193         assert "reel.mp4" in r.headers.get("content-disposition", "")
194

```

17. assistant (2026-06-18T07:55:12.019Z)

Now the async jobs and job-worker-loop tests, which cover the worker seam functions in main.py.

18. user (2026-06-18T07:55:12.820Z)

```

1     """Async job model (DEFERRED_HARDENING_PLAN #16).
2
3     Contract under test:
4     - POST /api/process ? 202 + job_id (fast-fail 4xx still happens at submit time)
5     - GET /api/jobs/{id} ? {status, progress, result, reports, ...}
6     - job `status` = EXECUTION state (queued|running|done|failed); the pipeline's own
7       verdict (e.g. failed validation) lives in result["status"].
8     - worker crash restores pre-run segments (destroy-AFTER-confirm) and marks the job
failed
9     - startup sweep (fail_stale_jobs) terminalizes jobs orphaned by a restart
10    - migration: fresh DB lands at user_version 2 with a jobs table; v1 DBs upgrade in place
11    """
12    import sqlite3
13    import time
14    from concurrent.futures import ThreadPoolExecutor
15
16    import pytest
17
18    pytest.importorskip("httpx") # TestClient needs httpx; skip cleanly where absent
19    from fastapi.testclient import TestClient
20
21    import backend.main as m
22    from backend.infrastructure.database import Database
23    from backend.config.models import AppConfig
24    from backend.pipeline.validators.base import ValidationResult
25
26
27    class _InlineExecutor:
28        """submit() runs the fn synchronously ? deterministic tests, no threads."""
29
30        def submit(self, fn, *args, **kwargs):
31            fn(*args, **kwargs)
32
33
34    @pytest.fixture
35    def env(tmp_path, monkeypatch):
36        db = Database(str(tmp_path / "t.db"))
37        cfg = AppConfig()
38        monkeypatch.setattr(m, "db_instance", db)
39        monkeypatch.setattr(m, "global_config", cfg)
40        monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
41        client = TestClient(m.app)
42        return client, db, tmp_path, monkeypatch
43
44
45    def _video(tmp_path, name="m.mp4"):
46        p = tmp_path / name
47        p.write_bytes(b"not-a-real-video-but-hashable")

```

```

48         return p
49
50
51     def _pass():
52         return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]
53
54
55     def _fail():
56         return [ValidationResult(validator_name="blur", passed=False, message="blurry",
57 details={})]
58
59     class _FakeSeg:
60         def __init__(self, db, cfg):
61             self.db = db
62
63         def process_video(self, video_id, video_path, *a, **k):
64             sid = self.db.add_play_segment(video_id, 0.0, 5.0)
65             return [{"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0,
66 "metadata": {}}]
67
68     class _RaisingSeg:
69         def __init__(self, db, cfg):
70             self.db = db
71
72         def process_video(self, video_id, video_path, *a, **k):
73             self.db.add_play_segment(video_id, 9.0, 10.0) # half-written row pre-crash
74             raise RuntimeError("boom")
75
76
77     # --- migration -----
78
79     def test_migration_fresh_db_is_v3_with_jobs_table(tmp_path):
80         db = Database(str(tmp_path / "fresh.db"))
81         with db._get_connection() as conn:
82             # v5 = per-user model store (PERSONALIZATION_PLAN.md §2); the jobs table arrived
83             assert conn.execute("PRAGMA user_version").fetchone()[0] == 5
84             cols = {r["name"] for r in conn.execute("PRAGMA table_info(jobs)").fetchall()}
85             assert {"id", "video_path", "video_id", "status", "progress", "error", "result"} <=
86 cols
87             assert {"policy", "next_poll_at"} <= cols # v3 self-scheduling columns
88
89     def test_migration_upgrades_v1_db_in_place(tmp_path):
90         path = str(tmp_path / "v1.db")
91         conn = sqlite3.connect(path)
92         conn.execute(
93             "CREATE TABLE videos (id TEXT PRIMARY KEY, filepath TEXT NOT NULL, "
94             "processed_at TIMESTAMP, status TEXT NOT NULL, validation_results TEXT)")
95         # A genuine v1 DB also has candidate_segments (created in the version<1 block);
96         include it
97         # so the v4 personalization ALTER (which adds user_id to that table) has a table to
98         alter.
99         conn.execute(
100             "CREATE TABLE candidate_segments (id INTEGER PRIMARY KEY AUTOINCREMENT, "
101             "video_id TEXT NOT NULL, detector TEXT NOT NULL, start_time REAL NOT NULL, "
102             "end_time REAL NOT NULL, duration REAL NOT NULL, stats TEXT)")
103         conn.execute("PRAGMA user_version = 1")
104         conn.commit()
105         conn.close()
106
107         db = Database(path) # ladder should take it 1 ? 5
108         with db._get_connection() as c:

```



```

107         assert c.execute("PRAGMA user_version").fetchone()[0] == 5
108         assert c.execute(
109             "SELECT name FROM sqlite_master WHERE type='table' AND
name='jobs']").fetchone()
110         # Restore v5 personalization coverage (user_models table + user_id on
candidate_segments).
111         # These were dropped in prior edit; add (do not replace) per review.
112         tables = {r[0] for r in c.execute(
113             "SELECT name FROM sqlite_master WHERE type='table'").fetchall()}
114         assert "user_models" in tables
115         cand_cols = {r["name"] for r in c.execute(
116             "PRAGMA table_info(candidate_segments)").fetchall()}
117         assert "user_id" in cand_cols
118
119
120     def test_parity_job_model_with_direct_processing(env, tmp_path, monkeypatch):
121         """Item 5 positive: direct CLI-style path (m._execute_processing) vs job-worker path
122         (submit + _InlineExecutor leading to _run_claimed_job / drain, plus explicit
_run_claimed_job)
123         produce equivalent outcomes for the same ProcessRequest shape. Exercises success,
124         Failure, and empty-run branches. Real DB queries using vid (no self-compares or
tautologies).
125         """
126         client, db, tpath, mp = env
127         vid_file = _video(tpath)
128         vid = m.compute_video_id(str(vid_file))
129         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
130         mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
131
132         req = m.ProcessRequest(video_path=str(vid_file), segmenter="x")
133
134         # Direct CLI-style path
135         direct_out = m._execute_processing(req)
136         segs = db.query_play_segments(vid, {})
137         assert len(segs) >= 1, f"direct path must affect segments for vid {vid}"
138         assert direct_out.get("result", {}).get("status") in (None, "success", "processed")
or "play_segments" in str(direct_out)
139
140         # Job-worker path via public API submit (exercises job creation + inline executor +
internal _run_claimed_job)
141         rj = client.post("/api/process", json={"video_path": str(vid_file), "segmenter":
"x"})
142         job_out = client.get(f"/api/jobs/{rj.json()['job_id']}").json()
143         segs_j = db.query_play_segments(vid, {})
144         assert job_out["result"]["status"] == "success"
145         assert len(segs_j) >= 1
146
147         # Explicit job-worker via _run_claimed_job (create + mark running + drive the
claimed runner)
148         vid_file2 = _video(tpath, name="m2.mp4")
149         vid2 = m.compute_video_id(str(vid_file2))
150         jid = f"parity-{vid2[:8]}"
151         db.create_job(jid, str(vid_file2))
152         db.mark_job_running(jid)
153         job_row = db.get_job(jid)
154         m._run_claimed_job(job_row)
155         segs2 = db.query_play_segments(vid2, {})
156         assert len(segs2) >= 1, f"_run_claimed_job path must produce segments for vid
{vid2}"
157
158         # Failure branch (direct + job submit)
159         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_fail(), {}))
160         req_f = m.ProcessRequest(video_path=str(vid_file))
161         direct_f = m._execute_processing(req_f)
162         rf = client.post("/api/process", json={"video_path": str(vid_file)})

```

```

163         jobf = client.get(f"/api/jobs/{rf.json()['job_id']}").json()
164         assert direct_f.get("result", {}).get("status") == "failed" or
jobf["result"]["status"] == "failed"
165
166         # Empty-run branch (success verdict but zero play_segments from this execution)
167         class _EmptySeg:
168             def __init__(self, db, cfg):
169                 self.db = db
170             def process_video(self, video_id, video_path, *a, **k):
171                 return []
172         mp.setattr(m.segmenter_registry, "get", lambda name: _EmptySeg)
173         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
174         req_e = m.ProcessRequest(video_path=str(vid_file))
175         de = m._execute_processing(req_e)
176         re = client.post("/api/process", json={"video_path": str(vid_file)})
177         je = client.get(f"/api/jobs/{re.json()['job_id']}").json()
178         assert de.get("result", {}).get("status") in (None, "success")
179         assert je["result"]["status"] == "success"
180         # (reingest safety for prior on empty/fail covered in dedicated tests)
181
182
183         # --- submit-time fast-fail -----
184
185         def test_submit_returns_202_with_job_id(env):
186             client, db, tmp_path, mp = env
187             vid_file = _video(tmp_path)
188             mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
189             mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
190             r = client.post("/api/process", json={"video_path": str(vid_file), "segmenter":
"x"})
191             assert r.status_code == 202
192             body = r.json()
193             assert body["job_id"]
194             assert body["status"] == "queued"
195             assert body["poll"].endswith(body["job_id"])
196
197
198         def test_submit_rejects_null_byte_path(env):
199             client = env[0]
200             r = client.post("/api/process", json={"video_path": "a\x00b.mp4"})
201             assert r.status_code == 400
202
203
204
205         def test_submit_missing_file_is_404(env):
206             client, db, tmp_path, mp = env
207             r = client.post("/api/process", json={"video_path": str(tmp_path / "nope.mp4")})
208             assert r.status_code == 404
209
210
211         def test_submit_unknown_segmenter_400_and_no_job_row(env):
212             client, db, tmp_path, mp = env
213             vid_file = _video(tmp_path)
214             mp.setattr(m.segmenter_registry, "get", lambda name: None)
215             mp.setattr(m.segmenter_registry, "list_available", lambda: {"motion": "..."})
216             r = client.post("/api/process", json={"video_path": str(vid_file), "segmenter":
"nope"})
217             assert r.status_code == 400
218             assert "Available segmenters" in r.json()["detail"]
219             with db._get_connection() as c:
220                 assert c.execute("SELECT COUNT(*) FROM jobs").fetchone()[0] == 0
221
222
223         # --- end-to-end job lifecycle (inline executor = deterministic) -----
224

```

```

225     def test_job_done_with_segments_and_video_id(env):
226         client, db, tmp_path, mp = env
227         vid_file = _video(tmp_path)
228         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
229         mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
230
231         job_id = client.post(
232             "/api/process", json={"video_path": str(vid_file), "segmenter":
"x").json()["job_id"]
233         r = client.get(f"/api/jobs/{job_id}")
234         assert r.status_code == 200
235         body = r.json()
236         assert body["status"] == "done"
237         assert body["progress"] == "finished"
238         assert body["error"] is None
239         assert body["result"]["status"] == "success"
240         assert len(body["result"]["play_segments"]) == 1
241         assert "reports" in body # None without obsreport; key must exist per the contract
242
243         vid = m.compute_video_id(str(vid_file))
244         assert body["video_id"] == vid
245         assert len(db.query_play_segments(vid, {})) == 1
246         assert db.get_video(vid)["status"] == "processed"
247
248
249     def test_job_done_but_pipeline_verdict_failed_preserves_prior(env):
250         client, db, tmp_path, mp = env
251         vid_file = _video(tmp_path)
252         vid = m.compute_video_id(str(vid_file))
253         db.add_video(vid, str(vid_file), "processed", [])
254         db.add_play_segment(vid, 1.0, 2.0) # prior good result
255         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_fail(), {}))
256
257         job_id = client.post("/api/process", json={"video_path":
str(vid_file)}).json()["job_id"]
258         body = client.get(f"/api/jobs/{job_id}").json()
259         # Worker finished fine ? the PIPELINE verdict is the failure.
260         assert body["status"] == "done"
261         assert body["result"]["status"] == "failed"
262         # PR-C semantics survive the async move: prior segments intact.
263         assert len(db.query_play_segments(vid, {})) == 1
264
265
266     def test_worker_crash_marks_failed_and_restores_prior_segments(env):
267         client, db, tmp_path, mp = env
268         vid_file = _video(tmp_path)
269         vid = m.compute_video_id(str(vid_file))
270         db.add_video(vid, str(vid_file), "processed", [])
271         db.add_play_segment(vid, 1.0, 2.0) # prior good result
272         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
273         mp.setattr(m.segmenter_registry, "get", lambda name: _RaisingSeg)
274
275         job_id = client.post(
276             "/api/process", json={"video_path": str(vid_file), "segmenter":
"x").json()["job_id"]
277         body = client.get(f"/api/jobs/{job_id}").json()
278         assert body["status"] == "failed"
279         assert "RuntimeError" in body["error"]
280         assert body["result"] is None
281         # The half-written 9.0?10.0 row was pruned; the prior 1.0?2.0 row survives.
282         segs = db.query_play_segments(vid, {})
283         assert len(segs) == 1 and segs[0]["start_time"] == 1.0
284
285
286     def test_progress_stages_are_recorded(env):

```

```

287     client, db, tmp_path, mp = env
288     vid_file = _video(tmp_path)
289     mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
290     mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
291
292     stages = []
293     real = db.set_job_progress
294
295     def spy(job_id, progress):
296         stages.append(progress)
297         return real(job_id, progress)
298
299     mp.setattr(db, "set_job_progress", spy)
300     client.post("/api/process", json={"video_path": str(vid_file), "segmenter": "x"})
301     assert "hashing" in stages
302     assert any(s.startswith("segmenting") for s in stages)
303
304
305     # --- polling endpoint -----
306
307     def test_get_unknown_job_is_404(env):
308         client = env[0]
309         assert client.get("/api/jobs/deadbeef").status_code == 404
310
311
312     # --- crash recovery -----
313
314     def test_fail_stale_jobs_terminalizes_queued_and_running_only(env):
315         client, db, tmp_path, mp = env
316         db.create_job("q1", "/v.mp4")
317         db.create_job("r1", "/v.mp4")
318         db.mark_job_running("r1")
319         db.create_job("d1", "/v.mp4")
320         db.mark_job_done("d1", {"status": "success"})
321
322         swept = db.fail_stale_jobs("interrupted by server restart")
323         assert swept == 2
324         assert db.get_job("q1")["status"] == "failed"
325         assert db.get_job("r1")["status"] == "failed"
326         assert "restart" in db.get_job("q1")["error"]
327         assert db.get_job("d1")["status"] == "done" # terminal states untouched
328
329
330     # --- real executor (cross-thread SQLite/WAL sanity) -----
331
332     def test_real_executor_completes_job(env):
333         client, db, tmp_path, mp = env
334         vid_file = _video(tmp_path)
335         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
336         mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
337         pool = ThreadPoolExecutor(max_workers=1)
338         mp.setattr(m, "_get_executor", lambda: pool)
339         try:
340             job_id = client.post(
341                 "/api/process", json={"video_path": str(vid_file), "segmenter":
"x"}).json()["job_id"]
342             deadline = time.time() + 10.0
343             body = None
344             while time.time() < deadline:
345                 body = client.get(f"/api/jobs/{job_id}").json()
346                 if body["status"] in ("done", "failed"):
347                     break
348                 time.sleep(0.05)
349             assert body is not None and body["status"] == "done"
350             assert body["result"]["status"] == "success"

```

```

351         finally:
352             pool.shutdown(wait=True)
353

```

19. assistant (2026-06-18T07:55:15.914Z)

Now let me check the job-worker-loop and self-scheduling tests to confirm coverage of the drain/wake-timer seam, and check whether `run\_cli\_diagnose\_clip` / the CLI `main()` body has any test coverage at all.

20. user (2026-06-18T07:55:16.865Z)

```

1      """P3 worker self-scheduling loop (EXECUTION_POLICY.md P3).
2
3      The P2 slice added the DB primitives (`claim_due_job`, `set_job_waiting`,
4      `seconds_until_next_due`); this slice wires the `main.py` worker to them so the worker
5      DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job that raises
6      `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and re-claimed
when
7      due ? instead of the old one-shot `submit(_run_job, ...)`
8
9      Covered:
10     - `_drain_due_jobs` claims + runs queued jobs to a terminal state (no regression on the
11       existing sync/submit path ? same `done`/`failed`/verdict semantics);
12     - earliest-due ordering across queued + due-waiting;
13     - a `JobWaiting` parks the job (no terminal state) and a later drain (once due) resumes
it;
14     - not-yet-due parked jobs are left alone and a wake-timer is armed for them;
15     - the wake-scheduling decision (`_schedule_next_drain_if_waiting`) is asserted with the
16       timer seam stubbed ? no real waiting, no leaked threads.
17
18     The real-timer seam (`_arm_wake_timer`) is monkeypatched to a recorder in every test
that
19     can park a job, so nothing actually sleeps and no daemon timer outlives the test.
20     """
21     import sqlite3
22
23     import pytest
24
25     pytest.importorskip("httpx")
26     from fastapi.testclient import TestClient
27
28     import backend.main as m
29     from backend.config.models import AppConfig
30     from backend.infrastructure.database import Database
31     from backend.pipeline.validators.base import ValidationResult
32
33
34     class _InlineExecutor:
35         """submit() runs fn synchronously ? deterministic, single-threaded tests."""
36
37         def submit(self, fn, *args, **kwargs):
38             fn(*args, **kwargs)
39
40
41     @pytest.fixture
42     def env(tmp_path, monkeypatch):
43         db = Database(str(tmp_path / "t.db"))
44         cfg = AppConfig()
45         monkeypatch.setattr(m, "db_instance", db)
46         monkeypatch.setattr(m, "global_config", cfg)
47         monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
48         # Stub the only real-timer seam so a parked job never arms a live daemon timer;
tests
49         # that care about scheduling assert against this recorder instead.

```

```

50     armed: list = []
51     monkeypatch.setattr(m, "_arm_wake_timer", lambda delay: armed.append(delay))
52     client = TestClient(m.app)
53     return client, db, tmp_path, monkeypatch, armed
54
55
56 def _video(tmp_path, name="m.mp4"):
57     p = tmp_path / name
58     p.write_bytes(b"not-a-real-video-but-hashable")
59     return p
60
61
62 def _pass():
63     return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]
64
65
66 class _FakeSeg:
67     def __init__(self, db, cfg):
68         self.db = db
69
70     def process_video(self, video_id, video_path, *a, **k):
71         sid = self.db.add_play_segment(video_id, 0.0, 5.0)
72         return [{"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0,
73 "metadata": {}}]
74
75 def _wire_happy(mp, client_env_video):
76     mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
77     mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
78
79
80 # --- no-regression: the drain runs a queued job exactly like the old submit path
81 -----
82 def test_drain_runs_queued_job_to_done(env):
83     client, db, tmp_path, mp, _armed = env
84     vid_file = _video(tmp_path)
85     _wire_happy(mp, vid_file)
86
87     # Submit via the public endpoint (which now submits a drain tick, not _run_job).
88     job_id = client.post(
89         "/api/process", json={"video_path": str(vid_file), "segmenter":
90 "x"}).json()["job_id"]
91     body = client.get(f"/api/jobs/{job_id}").json()
92     assert body["status"] == "done"
93     assert body["result"]["status"] == "success"
94     assert len(body["result"]["play_segments"]) == 1
95     vid = m.compute_video_id(str(vid_file))
96     assert body["video_id"] == vid
97     assert len(db.query_play_segments(vid, {})) == 1
98
99 def test_drain_returns_count_and_drains_all_queued(env):
100     client, db, tmp_path, mp, _armed = env
101     _wire_happy(mp, None)
102     # Two queued rows created directly (no endpoint), distinct video_paths.
103     a, b = _video(tmp_path, "a.mp4"), _video(tmp_path, "b.mp4")
104     db.create_job("a", str(a), segmenter="x")
105     db.create_job("b", str(b), segmenter="x")
106
107     ran = m._drain_due_jobs()
108     assert ran == 2
109     assert db.get_job("a")["status"] == "done"
110     assert db.get_job("b")["status"] == "done"
111     assert m._drain_due_jobs() == 0 # nothing runnable left

```

```

112
113
114 def test_drain_empty_table_is_noop(env):
115     _client, _db, _tmp, _mp, _armed = env
116     assert m._drain_due_jobs() == 0
117
118
119 # --- WAIT_AND_RESUME: a JobWaiting parks the job, a later drain resumes it
-----
120
121 def test_jobwaiting_parks_job_then_resumes_when_due(env):
122     client, db, tmp_path, mp, armed = env
123     vid_file = _video(tmp_path)
124     mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
125
126     # First execution parks the job (a FUTURE due time so it does NOT resume within the
same
127     # drain tick); the resume completes it. A one-shot toggle models "wait, then finish
next
128     # time" without a real external op.
129     state = {"parked": False}
130
131     class _WaitOnceSeg:
132         def __init__(self, db, cfg):
133             self.db = db
134
135         def process_video(self, video_id, video_path, *a, **k):
136             if not state["parked"]:
137                 state["parked"] = True
138                 raise m.JobWaiting(delay_sec=3600, progress="waiting on gemini")
139             sid = self.db.add_play_segment(video_id, 0.0, 5.0)
140             return [{"id": sid, "start_time": 0.0, "end_time": 5.0,
141                     "duration": 5.0, "metadata": {}}]
142
143     mp.setattr(m.segmenter_registry, "get", lambda name: _WaitOnceSeg)
144
145     job_id = client.post(
146         "/api/process", json={"video_path": str(vid_file), "segmenter":
"x").json()["job_id"]
147
148     # After the first drain the job is PARKED, not terminal, and its progress persisted.
149     parked = db.get_job(job_id)
150     assert parked["status"] == "waiting"
151     assert parked["progress"] == "waiting on gemini"
152     assert parked["next_poll_at"] is not None
153     assert parked["result"] is None # not done, not failed
154     # A wake-timer was armed for the parked job (clamped to the cap).
155     assert armed and 0.0 < armed[-1] <= m._MAX_DRAIN_WAKE_SEC
156
157     # Simulate the due time arriving: re-park as due-now, then drain re-claims and
finishes.
158     db.set_job_waiting(job_id, delay_sec=0)
159     m._drain_due_jobs()
160     done = db.get_job(job_id)
161     assert done["status"] == "done"
162     assert done["result"]["status"] == "success"
163
164
165 def test_not_yet_due_waiting_job_is_left_and_wake_armed(env):
166     _client, db, _tmp, _mp, armed = env
167     db.create_job("j1", "/v.mp4")
168     db.set_job_waiting("j1", delay_sec=3600, progress="parked")
169
170     ran = m._drain_due_jobs()
171     assert ran == 0 # not due -> not claimed

```

```

172     assert db.get_job("j1")["status"] == "waiting"      # left parked
173     # A wake was scheduled for ~the due time (clamped to the cap).
174     assert armed and 0.0 < armed[-1] <= m._MAX_DRAIN_WAKE_SEC
175
176
177     def test_no_wake_armed_when_nothing_waiting(env):
178         client, db, tmp_path, mp, armed = env
179         vid_file = _video(tmp_path)
180         _wire_happy(mp, vid_file)
181         client.post("/api/process", json={"video_path": str(vid_file), "segmenter": "x"})
182         assert armed == [] # happy-path job terminalises; nothing parked -> no timer
183
184
185     # --- ordering: earliest-due first across queued + due-waiting
186     -----
187     def test_claim_orders_earliest_due_first(env):
188         _client, db, _tmp, _mp, _armed = env
189         # b is parked due-immediately; a is plain queued. Earliest COALESCE(next_poll_at,
190         # created_at) wins. Set b due in the past so it precedes a's created_at.
191         db.create_job("a", "/a.mp4")
192         db.create_job("b", "/b.mp4")
193         db.set_job_waiting("b", delay_sec=0)
194         order = []
195         while True:
196             job = db.claim_due_job()
197             if job is None:
198                 break
199             order.append(job["id"])
200         assert set(order) == {"a", "b"}
201
202
203     # --- seconds_until_next_due (the wake-delay computation)
204     -----
205     def test_seconds_until_next_due_none_when_no_waiting(tmp_path):
206         db = Database(str(tmp_path / "d.db"))
207         db.create_job("q", "/v.mp4") # queued, not waiting
208         assert db.seconds_until_next_due() is None
209
210
211     def test_seconds_until_next_due_zero_when_due(tmp_path):
212         db = Database(str(tmp_path / "d.db"))
213         db.create_job("j", "/v.mp4")
214         db.set_job_waiting("j", delay_sec=0)
215         assert db.seconds_until_next_due() == 0.0
216
217
218     def test_seconds_until_next_due_positive_for_future(tmp_path):
219         db = Database(str(tmp_path / "d.db"))
220         db.create_job("j", "/v.mp4")
221         db.set_job_waiting("j", delay_sec=120)
222         secs = db.seconds_until_next_due()
223         assert secs is not None and 60.0 < secs <= 120.0 # ~120s out, allowing clock slack
224
225
226     def test_seconds_until_next_due_picks_soonest(tmp_path):
227         db = Database(str(tmp_path / "d.db"))
228         db.create_job("far", "/a.mp4")
229         db.create_job("near", "/b.mp4")
230         db.set_job_waiting("far", delay_sec=3600)
231         db.set_job_waiting("near", delay_sec=30)
232         secs = db.seconds_until_next_due()
233         assert secs is not None and secs <= 30.0 # the nearer job dominates
234

```



```

235
236     # --- a crash during the drain is a normal failure (not a wedge)
-----
237
238     def test_drain_segmenter_crash_marks_failed(env):
239         _client, db, tmp_path, mp, _armed = env
240         vid_file = _video(tmp_path)
241         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
242
243         class _BoomSeg:
244             def __init__(self, db, cfg):
245                 self.db = db
246
247             def process_video(self, *a, **k):
248                 raise RuntimeError("kaboom")
249
250         mp.setattr(m.segmenter_registry, "get", lambda name: _BoomSeg)
251         db.create_job("j", str(vid_file), segmenter="x")
252         ran = m._drain_due_jobs()
253         assert ran == 1
254         assert db.get_job("j")["status"] == "failed"
255         assert "RuntimeError" in db.get_job("j")["error"]
256
257
258     # --- migration sanity (the columns the loop depends on exist)
-----
259
260     def test_waiting_columns_present(tmp_path):
261         db = Database(str(tmp_path / "fresh.db"))
262         with db._get_connection() as conn:
263             cols = {r["name"] for r in conn.execute("PRAGMA table_info(jobs)").fetchall()}
264         assert {"policy", "next_poll_at", "status", "progress"} <= cols
265         # status='waiting' is a value, not a column; assert the round-trip works.
266         db.create_job("j", "/v.mp4")
267         db.set_job_waiting("j", delay_sec=5)
268         assert db.get_job("j")["status"] == "waiting"
269
270
271     def test_fresh_db_is_v5(tmp_path):
272         db = Database(str(tmp_path / "v.db"))
273         with db._get_connection() as conn:
274             # Head of the migration ladder (bump when database.py adds a new `if version <
275             N`)).
276             assert conn.execute("PRAGMA user_version").fetchone()[0] == 5
277
278     def test_legacy_db_upgrades_to_head(tmp_path):
279         # A pre-migration legacy DB (only `videos`, user_version 0) must run the FULL
280         ordered
281         # migration ladder up to the current head (v5) without error ? this exercises the
282         v0?v5
283         # path including the v1 `candidate_segments` CREATE that the later v4 ALTER depends
284         on.
285         path = str(tmp_path / "legacy.db")
286         conn = sqlite3.connect(path)
287         conn.execute(
288             "CREATE TABLE videos (id TEXT PRIMARY KEY, filepath TEXT NOT NULL, "
289             "processed_at TIMESTAMP, status TEXT NOT NULL, validation_results TEXT)")
290         conn.execute("PRAGMA user_version = 0")
291         conn.commit()
292         conn.close()
293         db = Database(path)
294         with db._get_connection() as c:
295             assert c.execute("PRAGMA user_version").fetchone()[0] == 5

```

21. assistant (2026-06-18T07:55:20.108Z)

I now have strong coverage knowledge. Let me confirm whether `run\_cli\_diagnose\_clip` and the CLI `main()` body have any test coverage, and check for any other module that imports the helpers I'd consider moving (to assess import-cycle / wiring risk).

22. user (2026-06-18T07:55:24.809Z)

```
=== who imports backend.main ===
tools/generate_highlights.py
=== refs to main()/run_cli_diagnose anywhere ===
backend/eval/ablation.py
backend/eval/audio/e1_probe.py
backend/eval/calibrate_local.py
backend/eval/calibrate_wasb.py
backend/eval/distill_local.py
backend/eval/eval_partial_labels.py
backend/eval/export_wasb_segments.py
backend/eval/fusion_audio.py
backend/eval/fusion_compare.py
backend/eval/fusion_golden.py
backend/eval/gemini_refine.py
backend/eval/golden_fixtures.py
backend/eval/rally_seg_eval.py
backend/eval/rally_seq_proto.py
backend/eval/served_gate_a.py
backend/eval/serve_contrast.py
backend/main.py
backend/pipeline/detectors/wasb_infer.py
backend/tools/rally_slicer.py
scripts/nightly_regression.py
scripts/run_phase3_eval.py
scripts/run_process_and_stitch.py
tools/cleanup_caches.py
tools/generate_highlights.py
=== entry point (console_scripts) ===
pyproject.toml:1:# PEP 621 packaging for the badminton/sports highlight indexer (DEFERRED #7).
pyproject.toml:17:name = "badminton-highlight-indexer"
pyproject.toml:68:# `indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
pyproject.toml:69:indexer = "backend.main:main"
```

23. assistant (2026-06-18T07:55:27.754Z)

Let me check what `tools/generate\_highlights.py` imports from `backend.main` (to assess what's safe to move) and confirm there's no test exercising `run\_cli\_diagnose\_clip` or the CLI `main()` body.

24. user (2026-06-18T07:55:31.048Z)

```
=== generate_highlights imports from backend.main ===
86: # standalone ? backend.main does this, but this module is its own entrypoint. Without it
the
=== any test referencing run_cli_diagnose_clip or m.main( ===
tests/test_ablation.py:503: ablation.main()
tests/test_ablation.py:518: ablation.main()
tests/test_ablation.py:530: ablation.main()
tests/test_eval_harness.py:99: rc = run_eval.main(["--scaffold", "--annotations", ann])
tests/test_eval_harness.py:106: assert run_eval.main([]) == 2
tests/test_eval_harness.py:113: rc = run_eval.main(["--video-id", "abc", "--annotations",
ann,
tests/test_eval_harness.py:128: rc = run_eval.main(["--video-id", "vid1", "--annotations",
str(ann),
tests/test_eval_regression.py:100: rc = run_regression.main(["--video-id", "vid1", "--tier",
"file",
```

```

tests/test_eval_regression.py:117: rc = run_regression.main(["--video-id", "vid1", "--tier",
"file",
tests/test_eval_regression.py:129: rc = run_regression.main(["--video-id", "vid1", "--tier",
"file",
=== _execute_processing / _finalize_reingest referenced in tests ===
tests/test_async_jobs.py
tests/test_base.py

```

25. assistant (2026-06-18T07:55:35.432Z)

The `generate\_highlights.py` reference is only a comment, not an import. So `backend.main` has effectively no external importers ? it's an entry-point module. And there is **no test** exercising `run\_cli\_diagnose\_clip` or the CLI `main()` body. This is a critical coverage fact.

Let me confirm the `\_finalize\_reingest` reference in `test\_base.py` and check the upload-cfg / config models to finalize my assessment of the upload-helper extraction seam.

26. user (2026-06-18T07:55:37.475Z)

```

=== test_base.py _finalize_reingest usage ===
1:import pytest
3:from backend.pipeline.segmenters.base import BasePlaySegmenter, SegmenterRegistry
4:from backend.results import Success, Failure, Result # test the new contract
=== database.py method count / grouping markers ===
10: def __init__(self, db_path: str):
15: def _connect(self):
29: def _get_connection(self):
45: def _add_column_idempotent(cursor: sqlite3.Cursor, ddl: str) -> None:
62: def _init_db(self):
251: def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
276: def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
290: def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
311: def add_play_segment(self, video_id: str, start_time: float, end_time: float, server:
Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]] = None)
-> Optional[int]:
325: def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
338: def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
361: def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
375: def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
400: def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
409: def query_play_segments(self, video_id: str, filters: Dict[str, Any]) -> List[Dict[str,
Any]]:
459: def clear_video_data(self, video_id: str):
464: # --- Async job store (DEFERRED_HARDENING_PLAN #16) -----
468: def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
489: def mark_job_running(self, job_id: str) -> bool:
501: def set_job_progress(self, job_id: str, progress: str) -> bool:
512: def set_job_video_id(self, job_id: str, video_id: str) -> bool:
523: def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
538: def mark_job_failed(self, job_id: str, error: str) -> bool:
551: def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
563: # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2)
-----
564: def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
577: def set_job_waiting(self, job_id: str, delay_sec: float, progress: Optional[str] = None)
-> bool:
594: def seconds_until_next_due(self) -> Optional[float]:
611: def claim_due_job(self) -> Optional[Dict[str, Any]]:
636: def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
652: # --- Per-user model store (PERSONALIZATION_PLAN.md §2, v5) -----
660: def upsert_user_model(self, user_id: Optional[str], *, version: int = 1,
721: def get_active_user_model(self, user_id: Optional[str]) -> Optional[Dict[str, Any]]:

```

27. assistant (2026-06-18T07:55:40.019Z)

I now have a complete picture. Let me run the relevant test suites quickly to confirm the baseline is green (so I can honestly assess "tests would catch a regression"), then finalize.

28. user (2026-06-18T07:56:04.523Z)

```
..... [ 86%]
..... [100%]
83 passed in 19.35s
```

29. assistant (2026-06-18T07:56:12.610Z)

Baseline is green (83 tests). I have everything I need. Let me consolidate the analysis into proposals.

```
**Key coverage facts established:**
- `database.py` has dedicated, thorough coverage in `test_database.py` plus the job-store
  methods are exercised end-to-end in `test_async_jobs.py` / `test_job_worker_loop.py`. The
  migration ladder has explicit idempotency/upgrade tests.
- `main.py` API endpoints, upload helpers (`_upload_cfg`/`_abort_upload`), and the job-worker
  seam (`_drain_due_jobs`, `_run_claimed_job`, `_schedule_next_drain_if_waiting`,
  `_arm_wake_timer`) are all covered via `TestClient` + direct calls.
- **No external module imports `backend.main`** (the `generate_highlights.py` hit is a comment)
  ? it's a console-script entry point. This means a same-module-helper extraction has essentially
  zero import-cycle/wiring risk, but moving symbols to a *new* module risks breaking the tests
  that do `import backend.main as m` and reference `m._arm_wake_timer`, `m.JobWaiting`,
  `m._uploads`, `m._execute_processing`, `m._run_claimed_job`, `m.ProcessRequest`, etc.
- **`run_cli_diagnose_clip` and the CLI `main()` body have ZERO test coverage** (cv2-dependent,
  can't run in this container). Anything touching them is "needs tests first."
```

30. user (2026-06-18T07:57:07.019Z)

Structured output provided successfully

31. assistant (2026-06-18T07:57:26.905Z)

I've completed the read-only analysis of the `core` cluster and returned five scoped proposals via StructuredOutput.

Summary of what I found and recommended:

```
**Safe, high-value picks (do these):**
- **`database.py::init_db` extraction (low risk, strong tests)** ? the 187-line method splits
  cleanly into per-table CREATE + per-version migration helpers, entirely in-class. The migration
  ladder has explicit idempotency/upgrade/version tests (`test_database.py`, plus migration tests
  in `test_async_jobs.py`/`test_job_worker_loop.py`) that would catch any regression. This is the
  cleanest win.
- **Extract chunked-upload endpoints into an APIRouter (high value, medium risk)** ? the
  ~130-line upload subsystem is the most cohesive island in `main.py` and is fully covered by
  `test_upload_download.py`. Medium (not low) because the tests monkeypatch `m._uploads` and read
  `global_config` off `backend.main`, so the move requires retargeting test patches and wiring the
  config global correctly.

**Medium / conditional:**
- **Extract the job-worker drain/wake loop** ? well-covered, but delicate: tests rely on
  `monkeypatch.setattr(m, '_arm_wake_timer', ...)` intercepting the *inter-function* call. A naive
  move would silently break that indirection and leak real daemon timers. Doable with a re-export
  shim, hence medium risk.
- **Dedup the ~14 repeated write-method try/except bodies in `database.py`** ? net-negative LOC,
  but tests assert on return values/status rather than on the per-method log strings, so it's
  partial coverage; the log-message text is the one thing a reviewer must confirm.
```

```
**Explicitly do NOT refactor yet (flagged needs-tests-first):**
```

- `main.py::main()` (168L) and `run\_cli\_diagnose\_clip` (cv2-driven) have **zero test coverage** and can't run in this container. Any change there is unverifiable locally ? high risk, low isolated value.

One important cross-cutting fact I verified: **no production module imports `backend.main`** (the one grep hit in `tools/generate\_highlights.py` is a comment), so it's purely a console-script entry point. The real blast radius for any `main.py` split is the test surface that does `import backend.main as m` and patches module globals ? which is why the upload/worker splits are medium rather than low. Baseline was green (83 tests across the touched suites) before I reported.

Relevant files:

- `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base/backend/main.py`
- `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base/backend/infrastructure/database.py`
- `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base/tests/test\_database.py`
- `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base/tests/test\_api\_endpoints.py`
- `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base/tests/test\_async\_jobs.py`
- `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base/tests/test\_job\_worker\_loop.py`
- `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base/tests/test\_upload\_download.py`